Université d'Évry – Paris-Saclay

Master 2 CNS – SR

Software Defined Network

Projet 02

Nom : Atsamnia
Prénom : Abderraouf
N° étudiant : 20264731
Parcours : CNS – SR

Octobre 2026

Table des matières

1 Introduction **3**

2 Exercice 1 – Topologie en arbre avec Mininet **3**

2.1 Question 1 – API de haut niveau 4

2.2 Question 2 – API de bas niveau 5

2.3 Question 3 – Connectivité et arrêt du switch cœur 6

3 Exercice 2 – Casser un réseau **8**

4 Exercice 3 – Messages OpenFlow entre Ryu et un switch **12**

4.1 Mise en place 12

4.2 Analyse des messages (Wireshark) 14

5 Exercice 4 – dpctl et ovs-dpctl **19**

5.1 Question 1 – Ping sans flux 20

5.2 Question 2 – Ping avec deux flux manuels 20

5.3 Question 3 – Statistiques des flux 22

5.4 ovs-dpctl 22

6 Exercice 5 – Découverte de topologie **23**

6.1 Topologie 23

6.2 OpenDaylight 24

6.3 ONOS 25

6.4 Question 2 – Ethertype et adresse MAC destination 26

6.5 Question 3 – TLV 27

6.6 Question 4 – Packet-In et Packet-Out 28

6.7 Question 5 – Fréquence d'envoi du LLDP 29

6.8 Question 6 – Comment le contrôleur découvre la topologie 31

7 Exercice 6 – Règles de commutation **31**

7.1 Deux switches en OpenFlow 1.3 31

7.2 Messages de type 14 33

7.3 Règles par défaut de s2 33

7.4 IDLE_TIMEOUT et HARD_TIMEOUT 34

7.5 Pings et nouveaux flux 34

7.6 Compteurs sur s1 et ping vers une adresse inexistante 36

8 Adaptations et difficultés **37**

9 Bilan **37**

10 Conclusion **37**

A Script `simpletree_highlevel.py` **38**

B Script `simpletree_lowlevel.py` **38**

C Script `loop.py` **39**

1 Introduction

Ce rapport présente le deuxième TP du module Software Defined Network. Il couvre six exercices : une topologie en arbre avec l'API de Mininet, un réseau avec une boucle, les messages OpenFlow 1.3 échangés avec le contrôleur Ryu, la manipulation directe des tables de flux, la découverte de topologie avec OpenDaylight et ONOS, et enfin l'analyse des règles de commutation installées par un contrôleur.

Pour chaque question du sujet, la réponse est donnée dans un encadré, avec les captures d'écran qui la justifient (S01 à S62). Deux captures manquent volontairement : S23 et S30. Elles correspondaient à des messages OpenFlow (`SET_CONFIG` et `SET_ASYNC`) que le contrôleur Ryu n'a pas envoyés, comme expliqué dans l'exercice 3.

Environnement utilisé.

- Machine hôte : Windows avec VMware Workstation.
- Machine virtuelle : image Mininet 2.3.0 sous Ubuntu 20.04.1 (4 Go de RAM, disque agrandi à 16 Go en cours de TP).
- Contrôleurs : Ryu (paquet `python3-ryu`), OpenDaylight Boron-SR3 (Java 8) et ONOS (conteneur Docker).
- Analyse du trafic : Wireshark avec le dissecteur OpenFlow.

Vérification de l'environnement au départ : versions de Python, de Mininet et d'Open vSwitch, et espace disque (figure 1).



Figure 1 – Versions de Python, Mininet, Open vSwitch et espace disque au départ.

2 Exercice 1 – Topologie en arbre avec Mininet

Pour une valeur n, la topologie contient un switch cœur, n switches d'agrégation, n^2 switches de bordure et n^3 hôtes. Pour $n = 2$, cela donne 7 switches et 8 hôtes. J'ai nommé les switches comme sur la figure du sujet (`c1`, `a1`, `e1`, ...), pour que la commande `switch c1 stop` de la question 3 fonctionne telle quelle. Chaque switch reçoit aussi un `dpid` explicite : sinon Mininet le déduit des chiffres du nom et `c1`, `a1` et `e1` auraient tous le même identifiant.

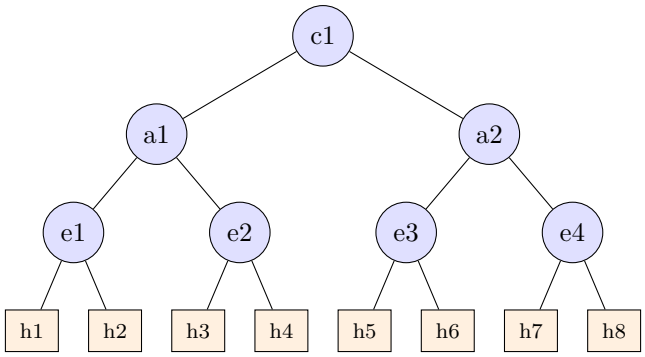


Figure 2 – Arbre simple pour $n = 2$.

2.1 Question 1 – API de haut niveau

Réponse

Le programme `simpletree_highlevel.py` (annexe A) définit une classe `SimpleTree` qui hérite de `Topo`. Sa méthode `build` crée le cœur, puis, pour chaque switch d'agrégation, ses n switches de bordure et leurs n hôtes. La valeur de n est passée en argument (ou demandée au clavier s'il est absent).

Le code est visible figure 3. Pour $n = 2$, Mininet crée bien 8 hôtes et 7 switches (figure 4). Avec $n = 3$, on obtient 27 hôtes et 13 switches (figure 5), ce qui confirme que la topologie dépend du paramètre.

mininet@mininet-vm: ~

```
mininet@mininet-vm:~$ cat ~/lab2/simpletree_highlevel.py
#!/usr/bin/env python3
import sys
from mininet.topo import Topo
from mininet.net import Mininet
from mininet.node import OVSSwitch
from mininet.cli import CLI
from mininet.log import setLogLevel


class SimpleTree(Topo):
    def build(self, n=2):
        dpid = 1
        core = self.addSwitch('c1', dpid='%x' % dpid)
        host_id = 1
        edge_id = 1
        for a in range(1, n + 1):
            dpid += 1
            agg = self.addSwitch('a%d' % a, dpid='%x' % dpid)
            self.addLink(core, agg)
            for _ in range(n):
                dpid += 1
                edge = self.addSwitch('e%d' % edge_id, dpid='%x' % dpid)
                edge_id += 1
                self.addLink(agg, edge)
                for _ in range(n):
                    host = self.addHost('h%d' % host_id)
                    host_id += 1
                    self.addLink(edge, host)


def run(n):
    net = Mininet(topo=SimpleTree(n), switch=OVSSwitch)
    net.start()
    CLI(net)
    net.stop()


if __name__ == '__main__':
    setLogLevel('info')
    n = int(sys.argv[1]) if len(sys.argv) > 1 else int(input('n = '))
    run(n)
mininet@mininet-vm:~$
```

Figure 3 – Programme `simpletree_highlevel.py`.

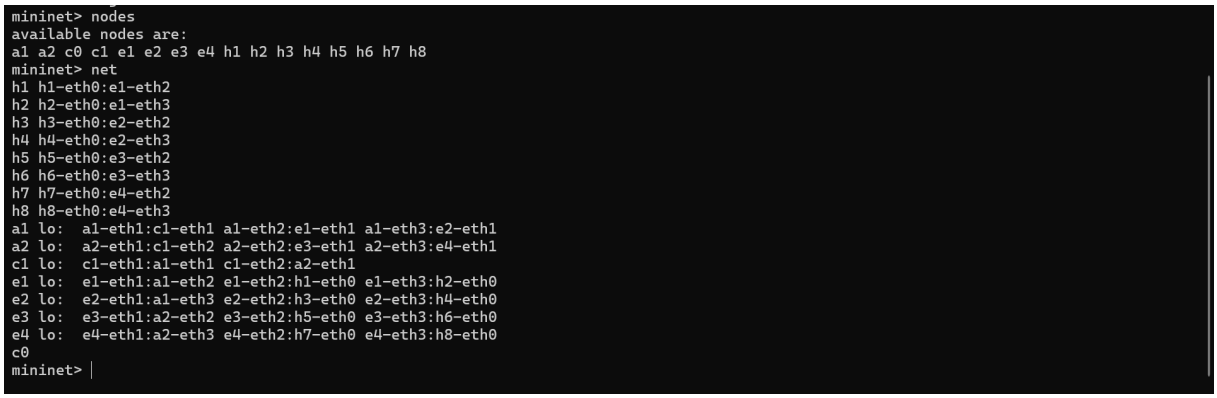


Figure 4 – Démarrage avec $n = 2$: commandes `nodes` et `net`.

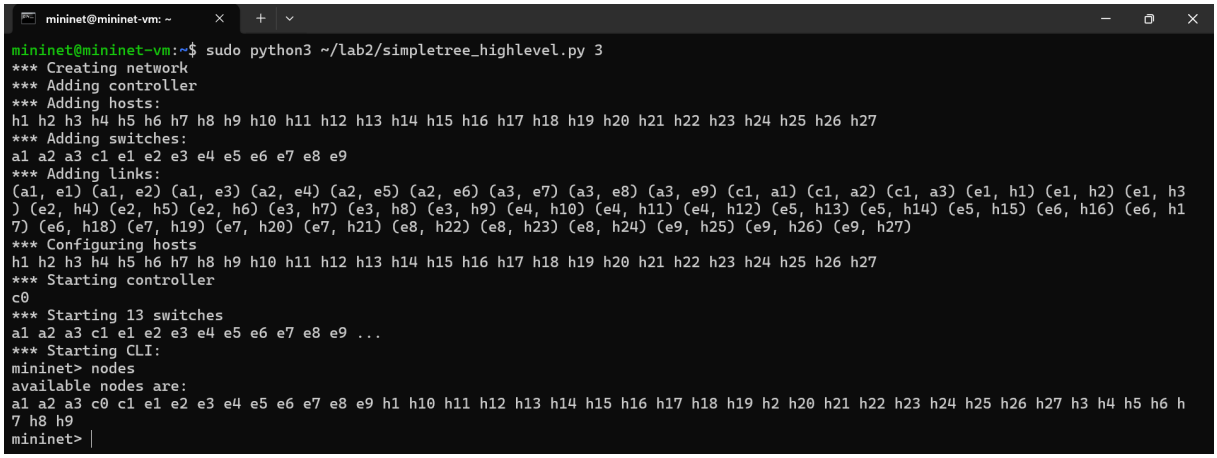
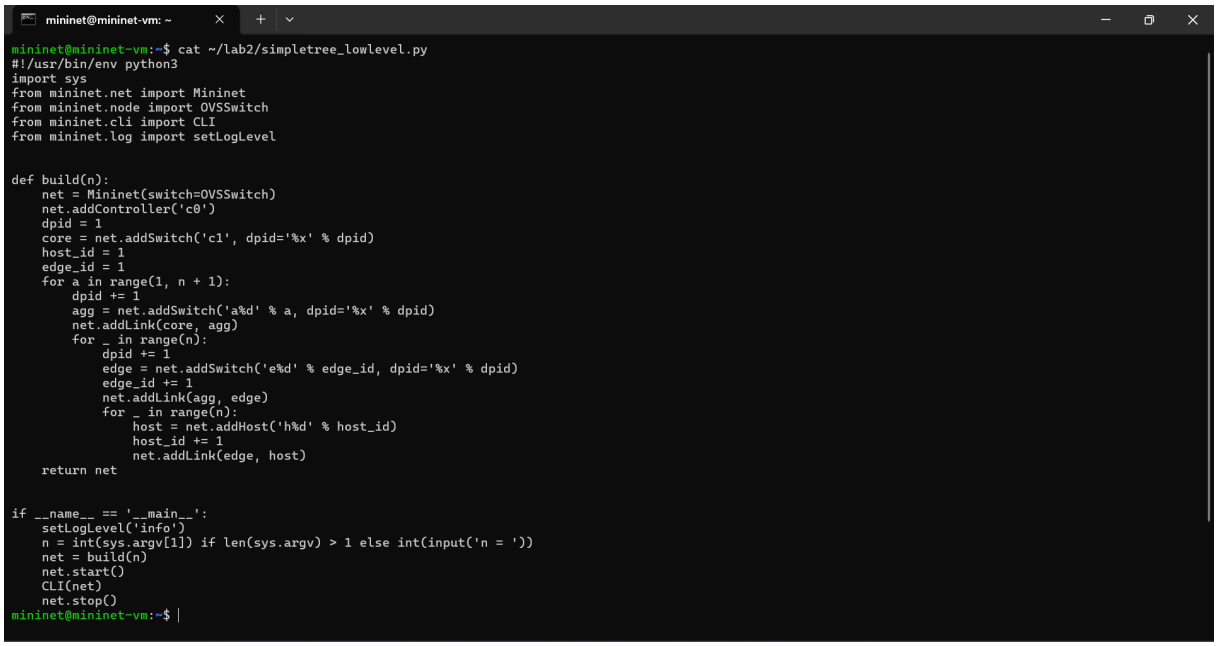


Figure 5 – Démarrage avec $n = 3$: 13 switches et 27 hôtes.

2.2 Question 2 – API de bas niveau

Réponse

Le programme `simpletree_lowlevel.py` (annexe B) construit la même topologie sans classe `Topo` : on crée l'objet `Mininet`, puis on appelle directement `addController`, `addSwitch`, `addHost` et `addLink`. Le résultat est identique à la version haut niveau.

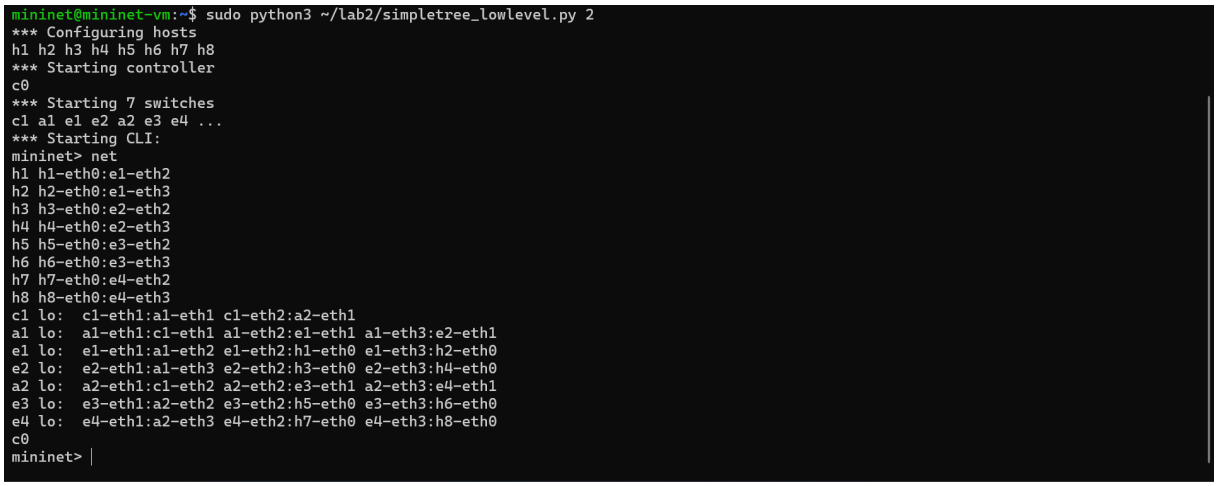


```
mininet@mininet-vm: ~
mininet@mininet-vm:~$ cat ~/lab2/simpletree_lowlevel.py
#!/usr/bin/env python3
import sys
from mininet.net import Mininet
from mininet.node import OVSSwitch
from mininet.cli import CLI
from mininet.log import setLogLevel


def build(n):
    net = Mininet(switch=OVSSwitch)
    net.addController('c0')
    dpid = 1
    core = net.addSwitch('c1', dpid='%x' % dpid)
    host_id = 1
    edge_id = 1
    for a in range(1, n + 1):
        dpid += 1
        agg = net.addSwitch('a%d' % a, dpid='%x' % dpid)
        net.addLink(core, agg)
        for _ in range(n):
            dpid += 1
            edge = net.addSwitch('e%d' % edge_id, dpid='%x' % dpid)
            edge_id += 1
            net.addLink(agg, edge)
            for _ in range(n):
                host = net.addHost('h%d' % host_id)
                host_id += 1
                net.addLink(edge, host)
    return net


if __name__ == '__main__':
    setLogLevel('info')
    n = int(sys.argv[1]) if len(sys.argv) > 1 else int(input('n = '))
    net = build(n)
    net.start()
    CLI(net)
    net.stop()
mininet@mininet-vm:~$
```

Figure 6 – Programme `simpletree_lowlevel.py`.



```
mininet@mininet-vm:~$ sudo python3 ~/lab2/simpletree_lowlevel.py 2
*** Configuring hosts
h1 h2 h3 h4 h5 h6 h7 h8
*** Starting controller
c0
*** Starting 7 switches
c1 a1 e1 e2 a2 e3 e4 ...
*** Starting CLI:
mininet> net
h1 h1-eth0:e1-eth2
h2 h2-eth0:e1-eth3
h3 h3-eth0:e2-eth2
h4 h4-eth0:e2-eth3
h5 h5-eth0:e3-eth2
h6 h6-eth0:e3-eth3
h7 h7-eth0:e4-eth2
h8 h8-eth0:e4-eth3
c1 lo:  c1-eth1:a1-eth1 c1-eth2:a2-eth1
a1 lo:  a1-eth1:c1-eth1 a1-eth2:e1-eth1 a1-eth3:e2-eth1
e1 lo:  e1-eth1:a1-eth2 e1-eth2:h1-eth0 e1-eth3:h2-eth0
e2 lo:  e2-eth1:a1-eth3 e2-eth2:h3-eth0 e2-eth3:h4-eth0
a2 lo:  a2-eth1:c1-eth2 a2-eth2:e3-eth1 a2-eth3:e4-eth1
e3 lo:  e3-eth1:a2-eth2 e3-eth2:h5-eth0 e3-eth3:h6-eth0
e4 lo:  e4-eth1:a2-eth3 e4-eth2:h7-eth0 e4-eth3:h8-eth0
c0
mininet>
```

Figure 7 – Topologie obtenue avec l'API bas niveau (commande `net`).

2.3 Question 3 – Connectivité et arrêt du switch cœur

Réponse

Avec $n = 2$, `pingall` ne perd aucun paquet (56 sur 56 reçus). Après `switch c1 stop`, environ 57 % des paquets sont perdus (32 sur 56) : les hôtes situés sous le même switch d'agrégation (h1 à h4 sous a1, h5 à h8 sous a2) se parlent toujours, mais plus aucun hôte d'une moitié ne joint l'autre. Le switch cœur est le seul lien entre les deux moitiés de l'arbre : c'est un point de défaillance unique, car l'arbre n'offre aucun chemin de secours.

Les résultats sont sur les figures 8 (avant l'arrêt), 9 (après l'arrêt) et 10 (pings ciblés : h1 vers h2 et h4 fonctionnent, h1 vers h5 échoue). En redémarrant c1 avec `switch c1 start`, la connectivité complète revient (figure 11).

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8
h2 -> h1 h3 h4 h5 h6 h7 h8
h3 -> h1 h2 h4 h5 h6 h7 h8
h4 -> h1 h2 h3 h5 h6 h7 h8
h5 -> h1 h2 h3 h4 h6 h7 h8
h6 -> h1 h2 h3 h4 h5 h7 h8
h7 -> h1 h2 h3 h4 h5 h6 h8
h8 -> h1 h2 h3 h4 h5 h6 h7
*** Results: 0% dropped (56/56 received)
mininet> h1 ping -c 3 h8
PING 10.0.0.8 (10.0.0.8) 56(84) bytes of data.
64 bytes from 10.0.0.8: icmp_seq=1 ttl=64 time=0.975 ms
64 bytes from 10.0.0.8: icmp_seq=2 ttl=64 time=0.151 ms
64 bytes from 10.0.0.8: icmp_seq=3 ttl=64 time=0.149 ms

--- 10.0.0.8 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 0.149/0.425/0.975/0.388 ms
mininet>
```

Figure 8 – `pingall` sans perte et ping de h1 vers h8 avant l'arrêt de c1.

```
mininet> switch c1 stop
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 X X X X
h2 -> h1 h3 h4 X X X X
h3 -> h1 h2 h4 X X X X
h4 -> h1 h2 h3 X X X X
h5 -> X X X X h6 h7 h8
h6 -> X X X X h5 h7 h8
h7 -> X X X X h5 h6 h8
h8 -> X X X X h5 h6 h7
*** Results: 57% dropped (24/56 received)
mininet>
```

Figure 9 – Arrêt du switch c1 puis `pingall` : environ 57 % de pertes.

```
mininet> h1 ping -c 3 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.23 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=4.09 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.365 ms

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 0.365/1.893/4.087/1.590 ms
mininet> h1 ping -c 3 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=2.39 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=2.88 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=0.305 ms

--- 10.0.0.4 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 0.305/1.860/2.882/1.117 ms
mininet> h1 ping -c 3 h5
PING 10.0.0.5 (10.0.0.5) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable
From 10.0.0.1 icmp_seq=3 Destination Host Unreachable

--- 10.0.0.5 ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2047ms
pipe 3
mininet>
```

Figure 10 – Après l'arrêt : h1 joint h2 et h4 mais pas h5.

```
mininet> switch c1 start
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8
h2 -> h1 h3 h4 h5 h6 h7 h8
h3 -> h1 h2 h4 h5 h6 h7 h8
h4 -> h1 h2 h3 h5 h6 h7 h8
h5 -> h1 h2 h3 h4 h6 h7 h8
h6 -> h1 h2 h3 h4 h5 h7 h8
h7 -> h1 h2 h3 h4 h5 h6 h8
h8 -> h1 h2 h3 h4 h5 h6 h7
*** Results: 0% dropped (56/56 received)
mininet>
```

Figure 11 – Redémarrage de c1 : la connectivité est rétablie.

3 Exercice 2 – Casser un réseau

J'ai construit un triangle de trois switches (s1, s2, s3), avec un hôte sur chacun, en utilisant `OVSBridge`. C'est un switch Ethernet autonome, sans contrôleur et sans spanning tree, comme les switches bon marché dont parle le sujet. Le script (`loop.py`, annexe C) construit le réseau puis ouvre le CLI. Le spanning tree est bien désactivé (figure 14).

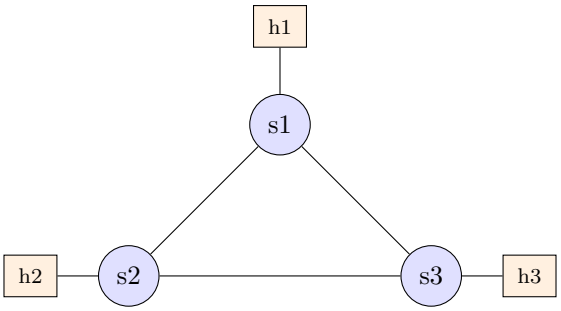


Figure 12 – Triangle de switches sans spanning tree.

mininet@mininet-vm: ~

```
mininet@mininet-vm:~$ cat ~/lab2/loop.py
#!/usr/bin/env python3
from mininet.net import Mininet
from mininet.node import OVSBridge
from mininet.cli import CLI
from mininet.log import setLogLevel


def run():
    net = Mininet(switch=OVSBridge, controller=None)
    s1 = net.addSwitch('s1')
    s2 = net.addSwitch('s2')
    s3 = net.addSwitch('s3')
    h1 = net.addHost('h1')
    h2 = net.addHost('h2')
    h3 = net.addHost('h3')
    net.addLink(h1, s1)
    net.addLink(h2, s2)
    net.addLink(h3, s3)
    net.addLink(s1, s2)
    net.addLink(s2, s3)
    net.addLink(s3, s1)
    net.start()
    CLI(net)
    net.stop()


if __name__ == '__main__':
    setLogLevel('info')
    run()
mininet@mininet-vm:~$
```

Figure 13 – Script `loop.py`.

```
mininet@mininet-vm:~$ sudo python3 ~/lab2/loop.py
*** Configuring hosts
h1 h2 h3
*** Starting controller

*** Starting 3 switches
s1 s2 s3 ...
*** Starting CLI:
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s2-eth1
h3 h3-eth0:s3-eth1
s1 lo:  s1-eth1:h1-eth0 s1-eth2:s2-eth2 s1-eth3:s3-eth3
s2 lo:  s2-eth1:h2-eth0 s2-eth2:s1-eth2 s2-eth3:s3-eth2
s3 lo:  s3-eth1:h3-eth0 s3-eth2:s2-eth3 s3-eth3:s1-eth3
mininet> sh ovs-vsctl get bridge s1 stp_enable
false
mininet>
```

Figure 14 – Topologie (commande `net`) et spanning tree désactivé.

Réponse

Les pings entre les hôtes échouent ou deviennent très instables (pertes, délais anormaux), et la machine ralentit nettement. La cause est une *tempête de broadcast*. Le premier ARP envoyé par un hôte est un broadcast : chaque switch le recopie sur tous ses autres ports, il fait donc le tour du triangle sans jamais s'arrêter. Une trame Ethernet n'a pas de TTL, et sans spanning tree aucun port n'est bloqué. Chaque tour multiplie les copies et les tables d'adresses MAC des switches changent sans cesse de port.

Les captures illustrent ce comportement : les pings (figure 15), la charge CPU relevée avec `top` (figure 16) et la capture Wireshark sur `s1-eth1` avec le filtre `arp` (figure 17). Cette dernière compte plus de 5,6 millions de trames ARP en une minute environ, soit de l'ordre de 100 000 trames par seconde, venant de deux ou trois hôtes seulement.

```
mininet>    h1 ping -c 5 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable
From 10.0.0.1 icmp_seq=3 Destination Host Unreachable
From 10.0.0.1 icmp_seq=4 Destination Host Unreachable
From 10.0.0.1 icmp_seq=5 Destination Host Unreachable

--- 10.0.0.2 ping statistics ---
5 packets transmitted, 0 received, +5 errors, 100% packet loss, time 4088ms
pipe 4
mininet> pingall
*** Ping: testing ping reachability
h1 -> X h3
h2 -> X X
h3 -> X X
*** Results: 83% dropped (1/6 received)
mininet>
```

Figure 15 – Pings entre les hôtes du triangle.

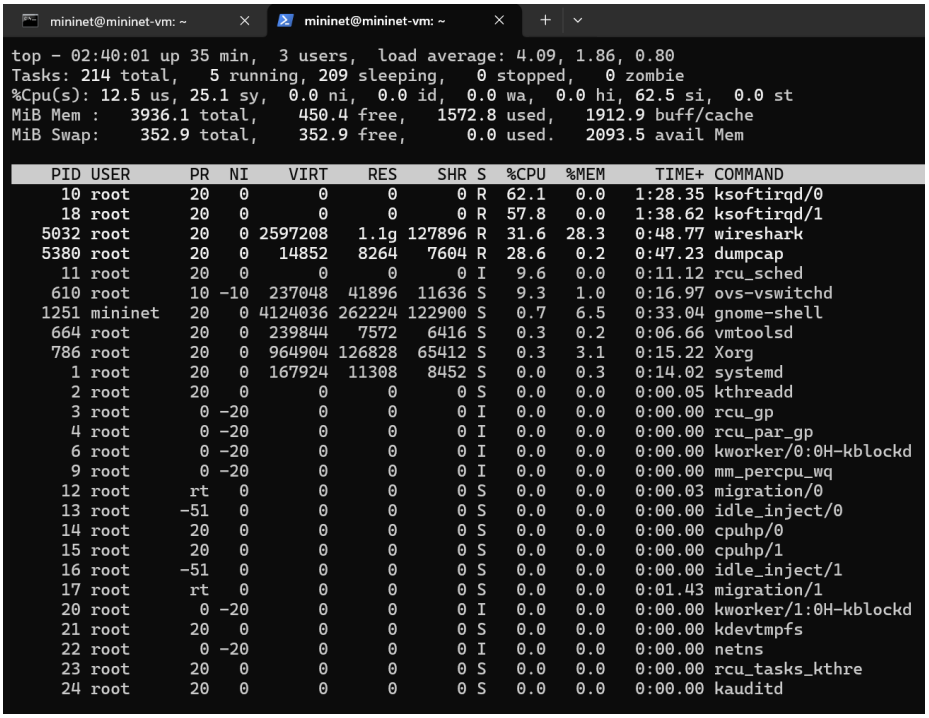


Figure 16 – Charge CPU élevée pendant la tempête de broadcast.

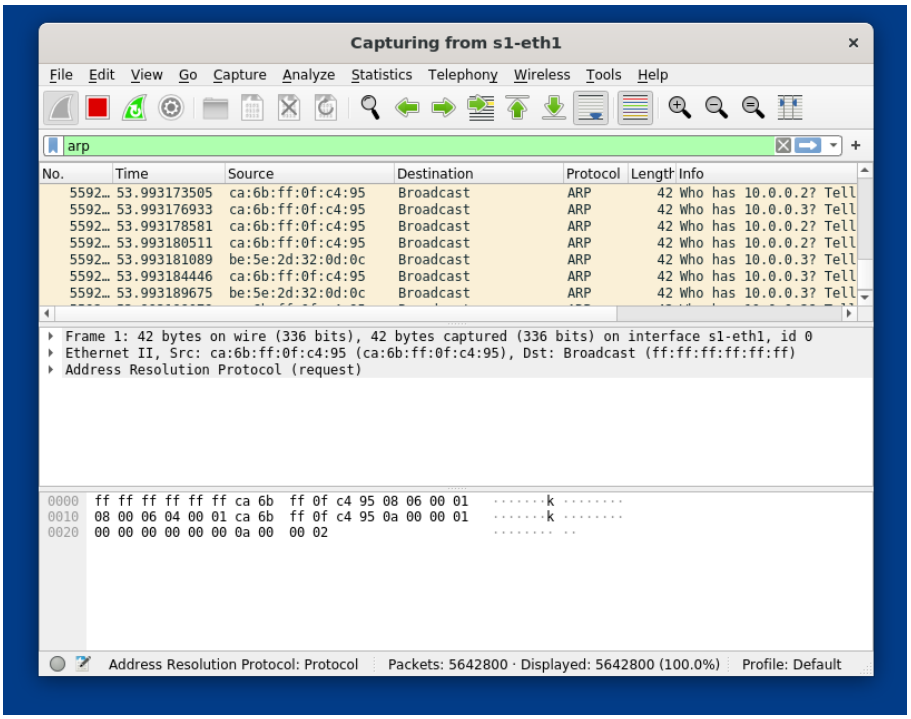


Figure 17 – Les mêmes requêtes ARP répétées en continu dans Wireshark.

4 Exercice 3 – Messages OpenFlow entre Ryu et un switch

4.1 Mise en place

Le sujet demande `python2-ryu`, qui n'existe plus sur cette VM (Python 3 uniquement). J'ai installé `python3-ryu` avec `apt-get` (figure 18). Comme Ryu est installé comme un paquet, je le lance avec `ryu-manager --verbose ryu.app.simple_switch_13`, et non avec le chemin `./bin/ryu-manager` du sujet.

Pour ne manquer aucun message, j'ai démarré Wireshark (interface `lo`, filtre `openflow_v4`) en premier, puis Ryu, puis Mininet :

```
sudo mn --topo single,3 --mac --controller remote --switch ovsk,protocols=OpenFlow13
```

Le journal de Ryu montre la connexion du switch et la fin de la négociation (`move onto main mode`, figure 19). Un ping entre h1 et h3 fonctionne (figure 20) et les flux installés sont visibles avec `ovs-ofctl dump-flows s1 -O OpenFlow13` (figure 21).

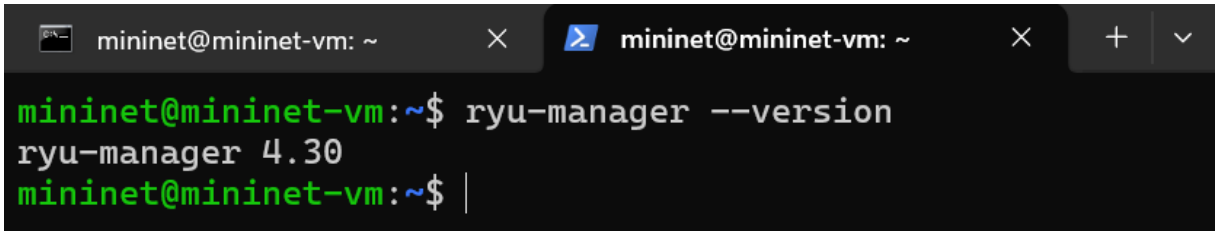


Figure 18 – Installation de Ryu : version du gestionnaire.

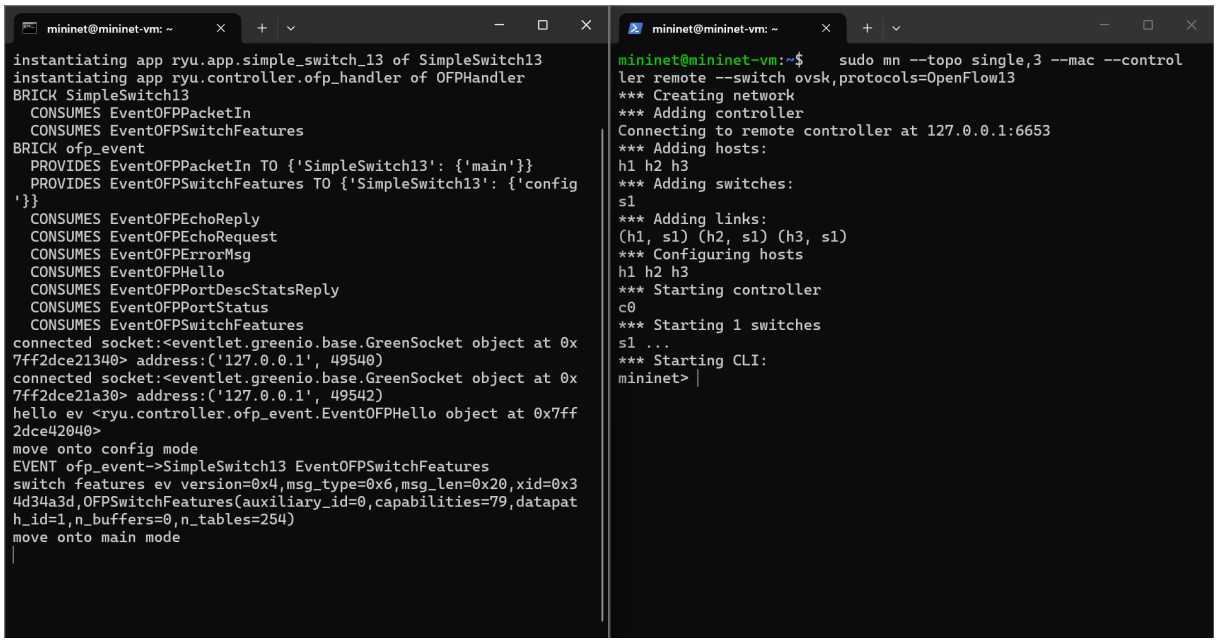


Figure 19 – Journal de Ryu avec le switch connecté (à gauche) et Mininet démarré.

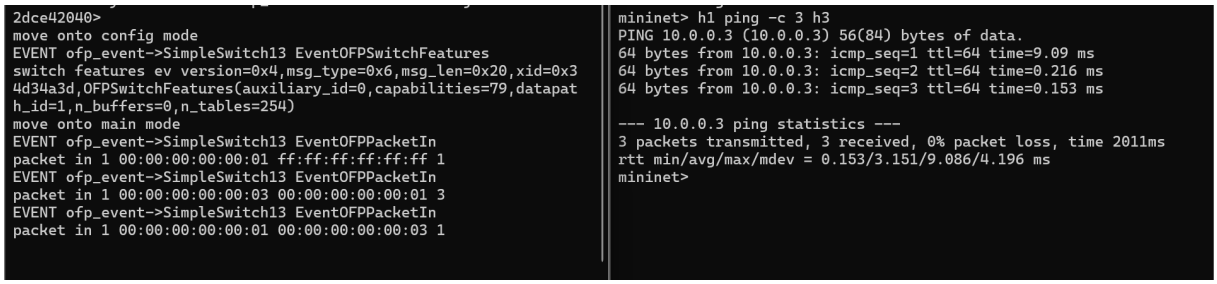


Figure 20 – Ping de h1 vers h3 : le premier est plus lent que les suivants.

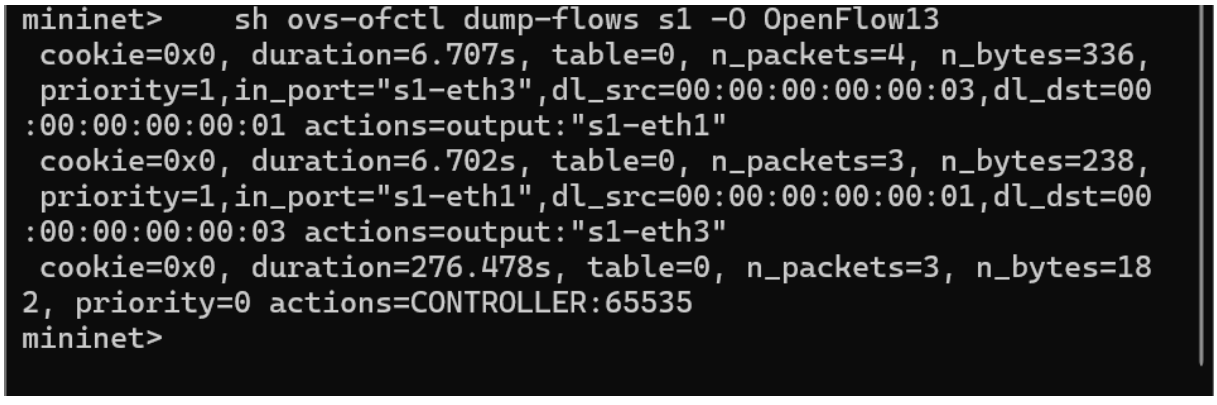


Figure 21 – Table de flux de s1 après le ping.

Les flux de la figure 21 sont : une règle `table-miss` de priorité 0 qui envoie au contrôleur tout paquet inconnu (3 paquets déjà traités), et deux règles de priorité 1 apprises par Ryu, de h1 vers h3 et de h3 vers h1, qui filtrent sur le port d'entrée et les adresses MAC source et destination. Les compteurs de ces deux règles correspondent aux paquets du ping.

4.2 Analyse des messages (Wireshark)

Réponse

Messages observés dans l'ordre : `HELLO`, `FEATURES_REQUEST` et `REPLY`, `MULTIPART` pour la description des ports, `PACKET_IN`, puis `FLOW_MOD`. Les messages `SET_CONFIG` et `SET_ASYNC` n'apparaissent pas avec ce contrôleur (voir plus bas).

Hello (S20). Chaque côté envoie un `OFPT_HELLO` avec sa version la plus haute. Le champ version vaut `0x04`, c'est-à-dire OpenFlow 1.3, qui est donc la version négociée.

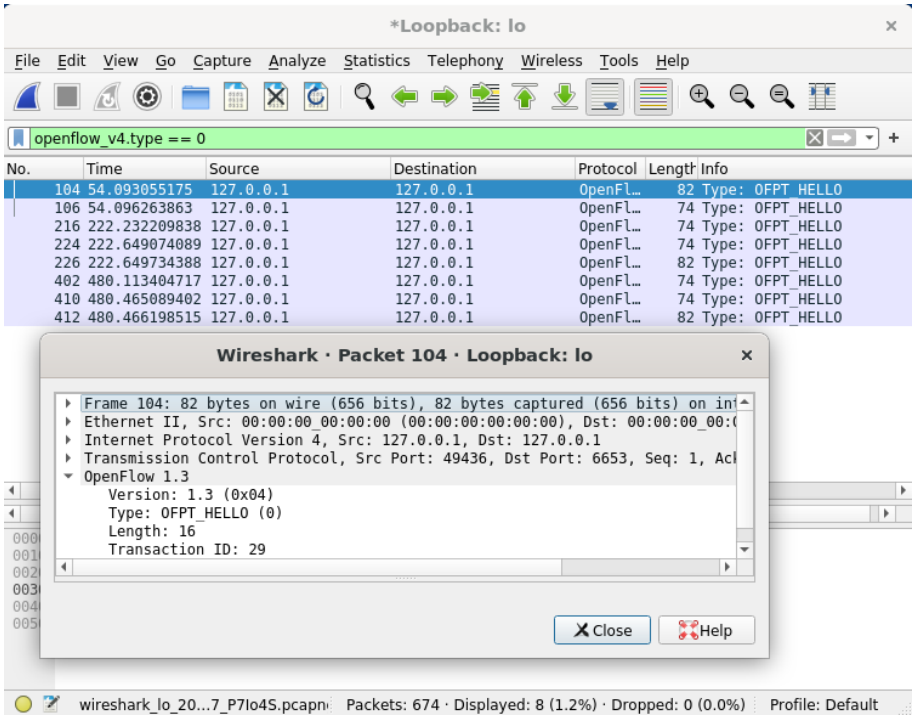


Figure 22 – Message `OFPT_HELLO` : version `0x04`.

Features request et reply (S21, S22). Le contrôleur envoie une requête qui ne contient qu'un en-tête. Le switch répond avec son identifiant de datapath (`0x1`, soit s1), le nombre de tables (254), un nombre de buffers de 0 et ses capacités (`0x4f` : statistiques de flux, de tables, de ports, de groupes et de files). Contrairement à OpenFlow 1.0, la réponse ne contient plus la liste des ports.

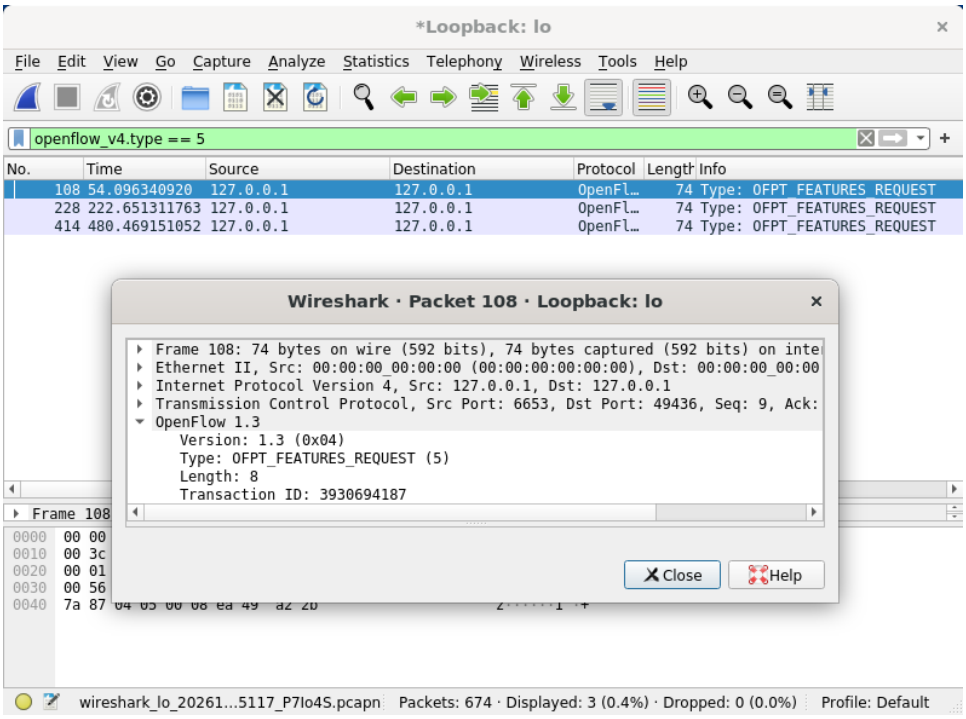


Figure 23 – `OFPT_FEATURES_REQUEST` : un simple en-tête.

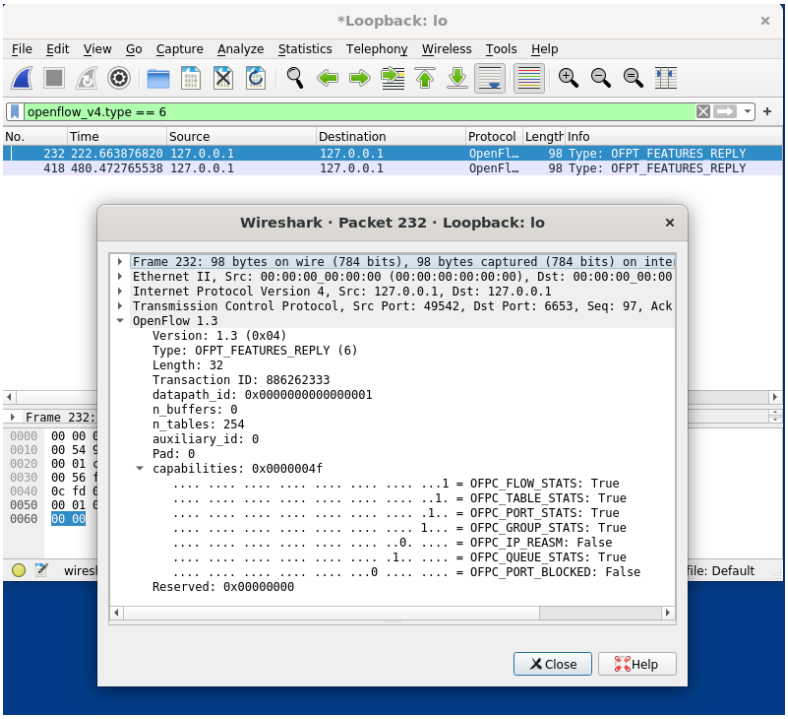


Figure 24 – `OFPT_FEATURES_REPLY` : identifiant du datapath, tables et capacités.

Set config (S23). Je n'ai pas de capture pour ce message : le filtre `openflow_v4.type == 9` ne renvoie rien. D'après le sujet, ce message transmet des drapeaux et le nombre maximal d'octets de chaque paquet à envoyer au contrôleur. Ici, ce n'est pas un problème : la règle `table-miss` utilise `CONTROLLER:65535`, donc le paquet complet est envoyé au contrôleur.

Multipart request et reply (S24, S25). Le contrôleur demande la description des ports avec un `OFPT_MULTIPART_REQUEST` de type `OFPMP_PORT_DESC`. Le switch répond avec un `OFPT_MULTIPART_REPLY` qui liste tous ses ports. En OpenFlow 1.3, cette information a donc quitté le message `FEATURES_REPLY`.

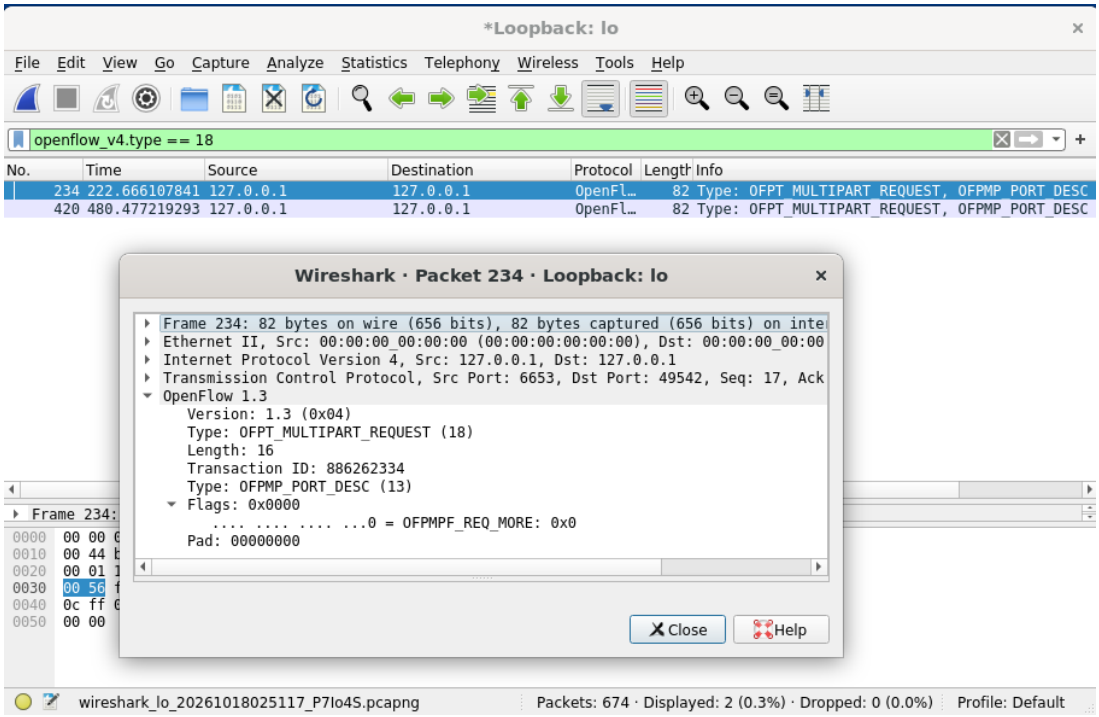


Figure 25 – `OFPT_MULTIPART_REQUEST` de type description des ports.

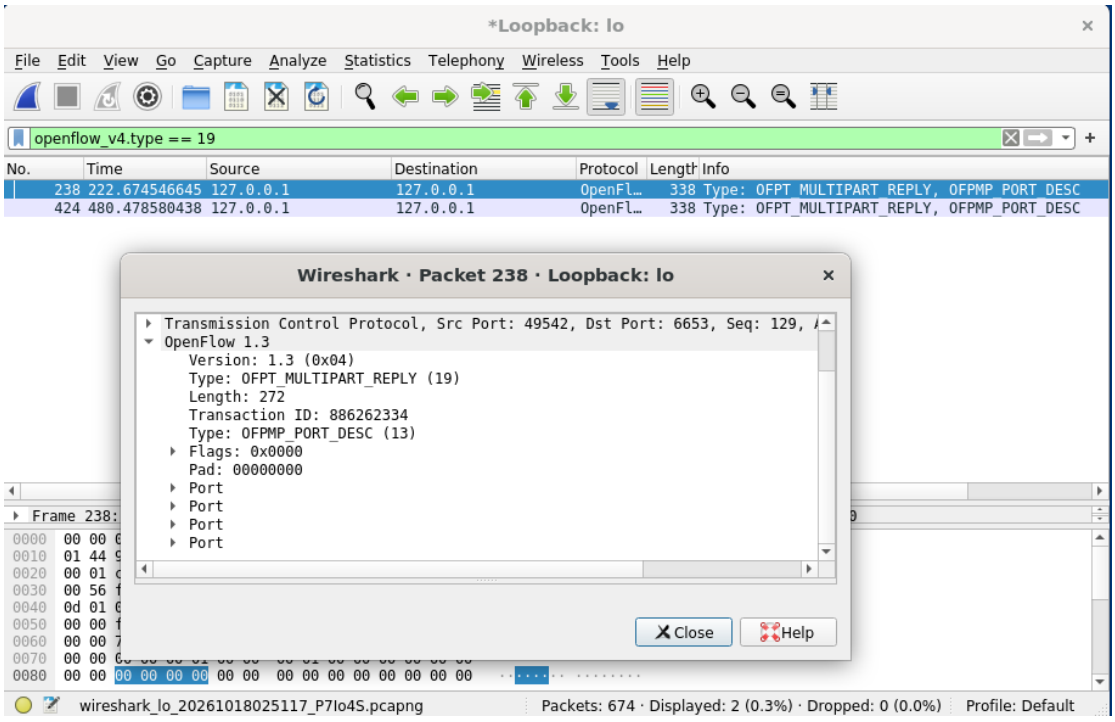


Figure 26 – `OFPT_MULTIPART_REPLY` avec les ports de s1.

Packet-In (S26). Quand h1 envoie son premier ARP, le switch n'a aucune règle correspondante. Il l'envoie au contrôleur dans un `OFPT_PACKET_IN`, avec comme raison l'absence de règle

(`table-miss`), le port d'entrée et le contenu du paquet.

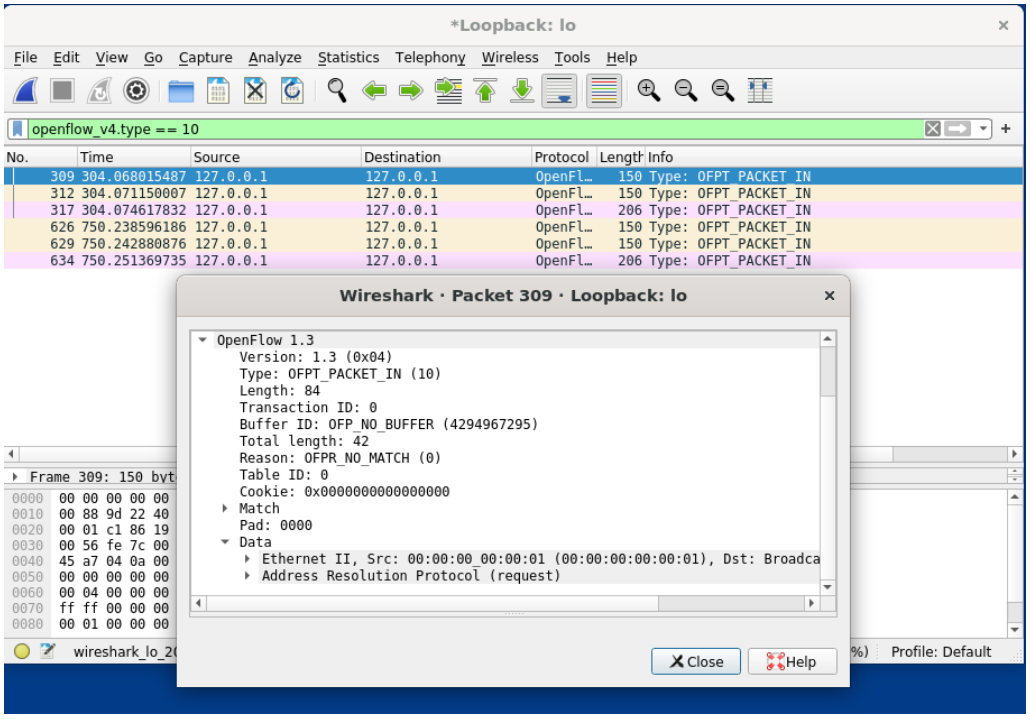


Figure 27 – `OFPT_PACKET_IN` : un ARP envoyé au contrôleur.

Flow-Mod (S27, S28). Le premier `OFPT_FLOW_MOD` installe la règle `table-miss` (commande `OFPFC_ADD`, priorité 0, match vide). Les suivants, après le ping, installent les règles apprises (priorité 1) avec des champs de match (port d'entrée, adresse MAC destination) et une instruction qui envoie le paquet sur un port. Le switch n'acquitte pas un `FLOW_MOD` ; en cas d'erreur, il répond par un message d'erreur.

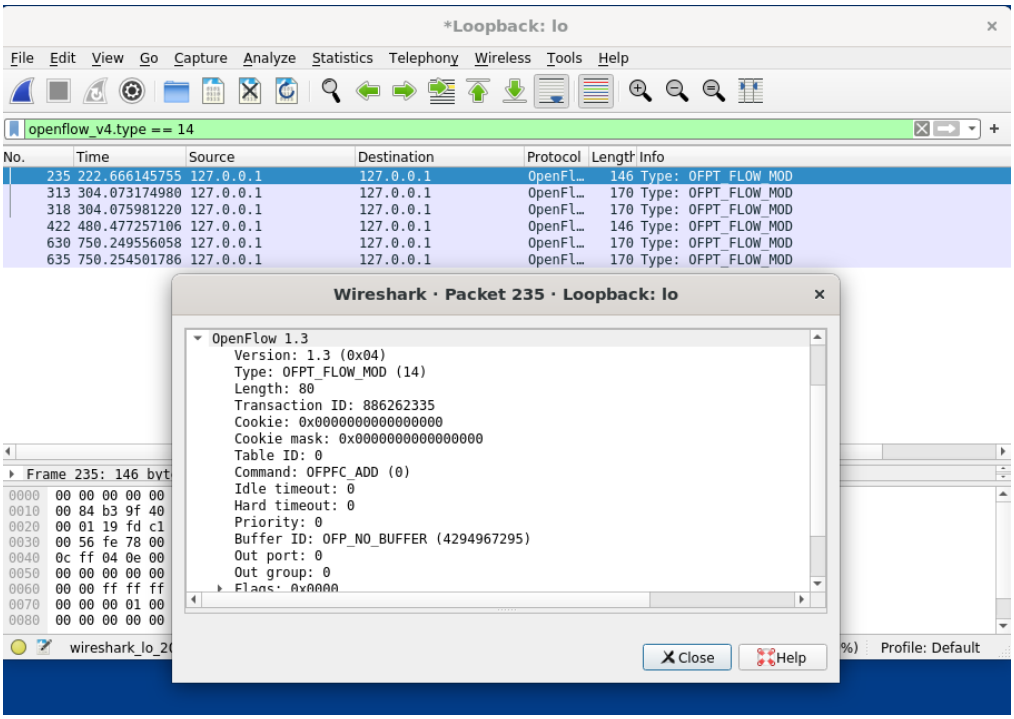


Figure 28 – Premier `FLOW_MOD` : règle `table-miss`, priorité 0.

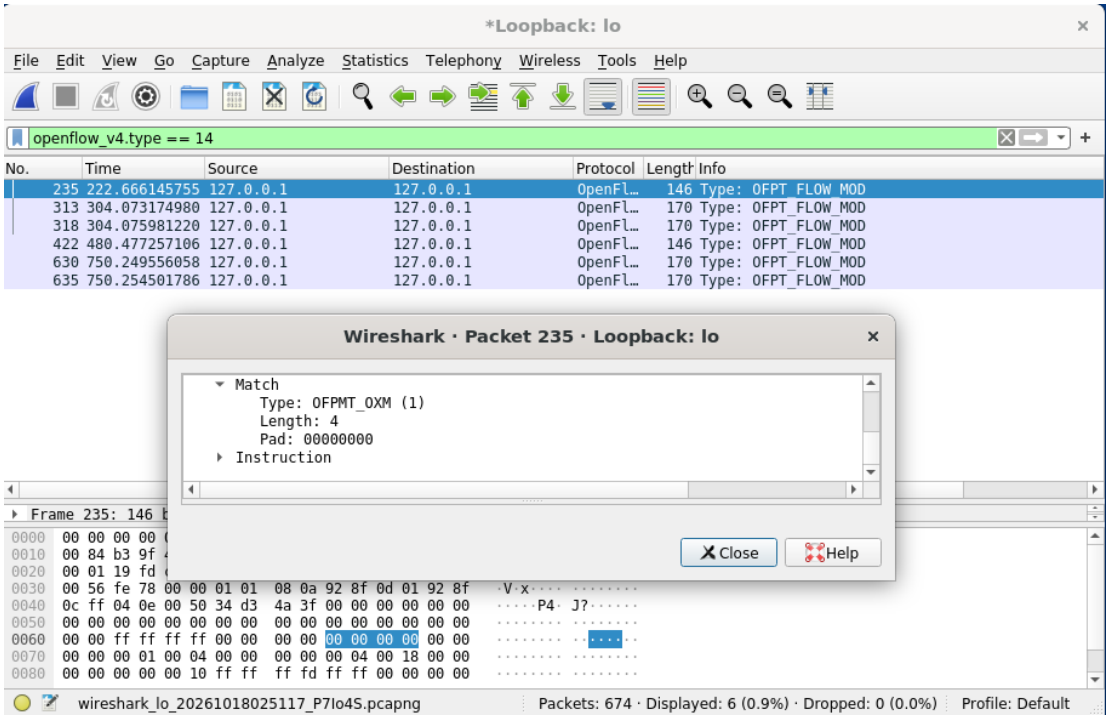


Figure 29 – `FLOW_MOD` appris : priorité 1, match et instruction.

Vue d'ensemble (S29). La liste complète des messages montre l'enchaînement de la négociation, puis des échanges liés au ping (`PACKET_IN`, `PACKET_OUT`, `FLOW_MOD`).

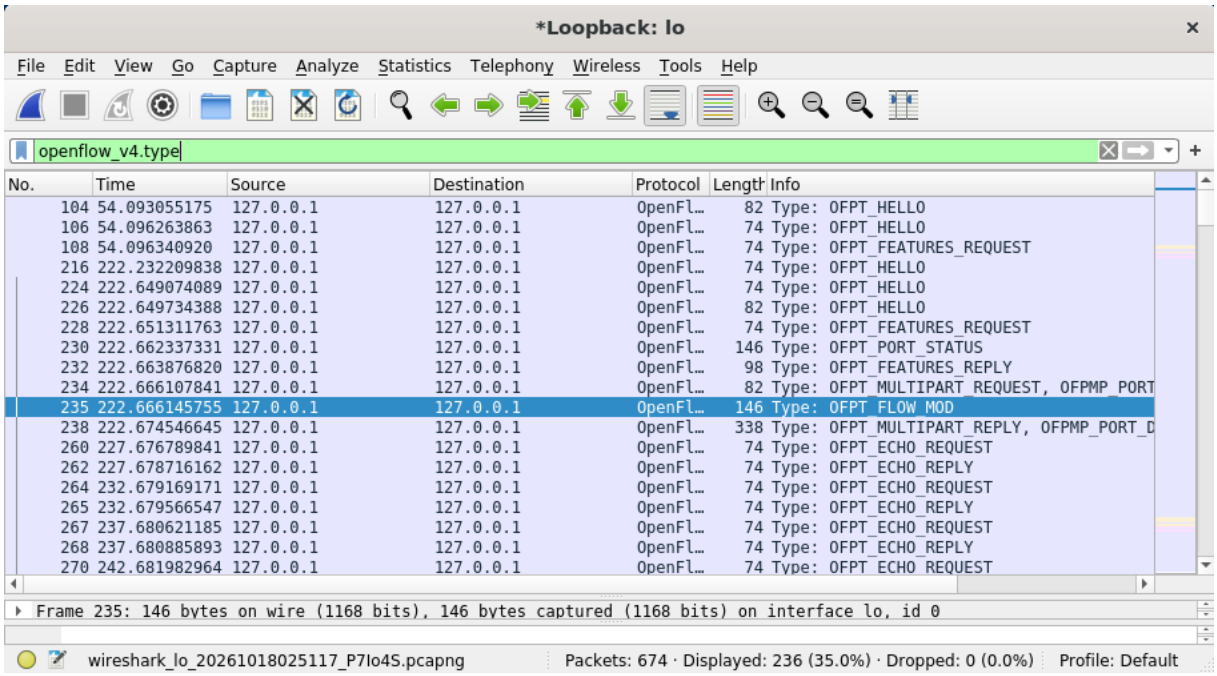


Figure 30 – Séquence complète des messages OpenFlow entre Ryu et le switch.

Set async (S30). Je n'ai pas non plus de capture de ce message : le filtre `openflow_v4.type == 28` ne renvoie rien. Ce message sert au contrôleur à choisir les messages asynchrones qu'il veut recevoir (Packet-In, Port-Status, Flow-Removed). La version de Ryu installée dans la VM ne l'envoie pas : le switch garde ses réglages par défaut.

5 Exercice 4 – dpctl et ovs-dpctl

La topologie est lancée sans aucun contrôleur :

```
sudo mn --topo single,3 --mac --switch ovsk --controller remote
```

Le sujet utilise `dpctl ... tcp:127.0.0.1:6634`, qui visait l'ancien switch noyau de Mininet. Avec Open vSwitch, le switch n'écoute pas sur ce port et la commande échoue. J'ai utilisé à la place `ovs-ofctl`, qui interroge le switch par un socket local (figure 31). Les ports sont : h1 sur le port 1, h2 sur le port 2 et h3 sur le port 3.

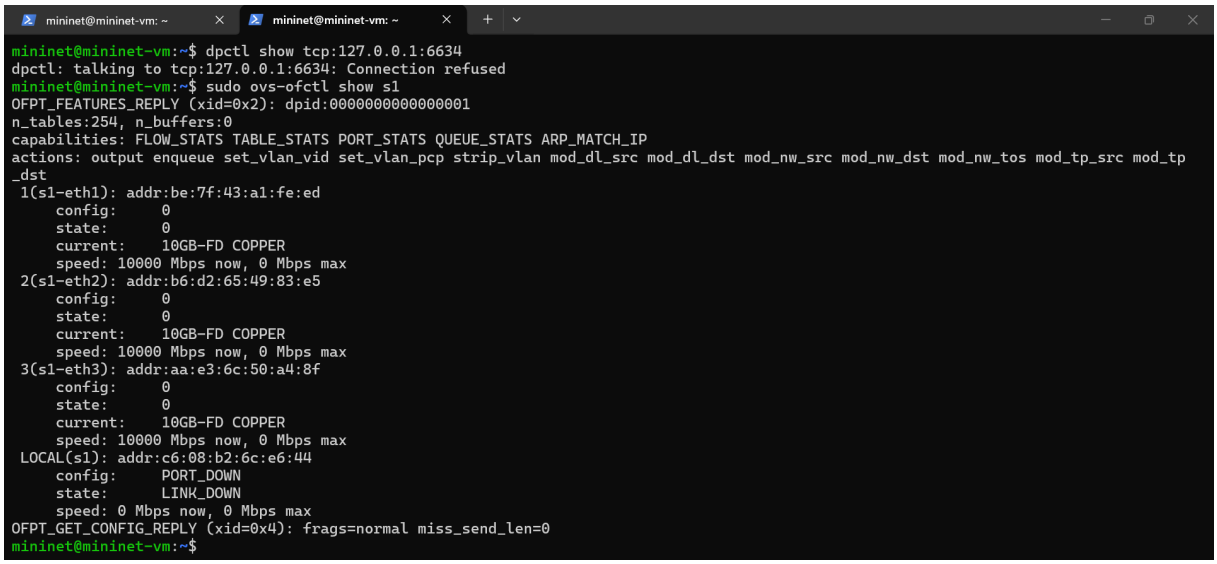


Figure 31 – Échec de `dpctl` sur le port 6634 et sortie de `ovs-ofctl show s1`.

5.1 Question 1 – Ping sans flux

Réponse

Non, h2 ne reçoit aucune réponse de h3. Le switch n'a aucune règle dans sa table de flux et aucun contrôleur ne lui indique quoi faire. Mininet le configure en mode « secure », où il abandonne tout ce qu'il ne sait pas traiter.

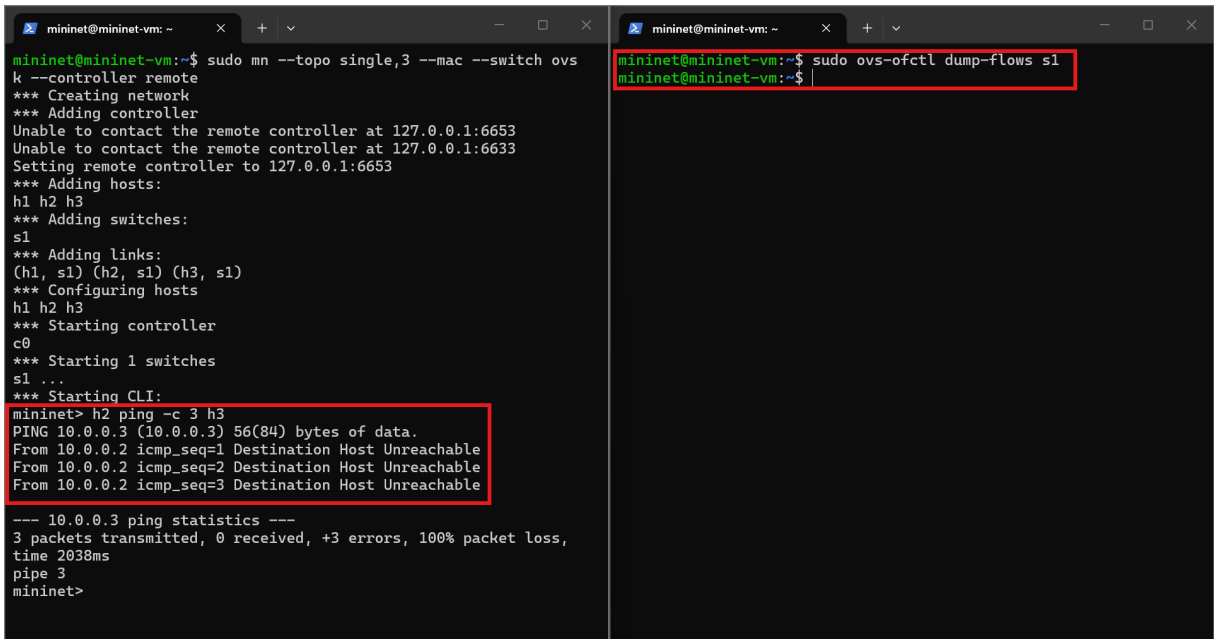


Figure 32 – Table de flux vide et ping de h2 vers h3 en échec.

5.2 Question 2 – Ping avec deux flux manuels

J'ai ajouté les deux flux du sujet avec `ovs-ofctl add-flow s1 in_port=1,actions=output:2` et son symétrique (figure 33).

Réponse

Le ping de h2 vers h3 échoue toujours, alors que celui de h1 vers h2 fonctionne. Les deux flux relient les ports 1 et 2, c'est-à-dire h1 et h2. Dans notre Mininet, h3 est sur le port 3 : il n'y a donc aucune règle vers lui. Le schéma du sujet nomme ses hôtes autrement (h2, h3 et h4 sur les ports 0 à 2), ce qui explique l'écart avec son exemple.

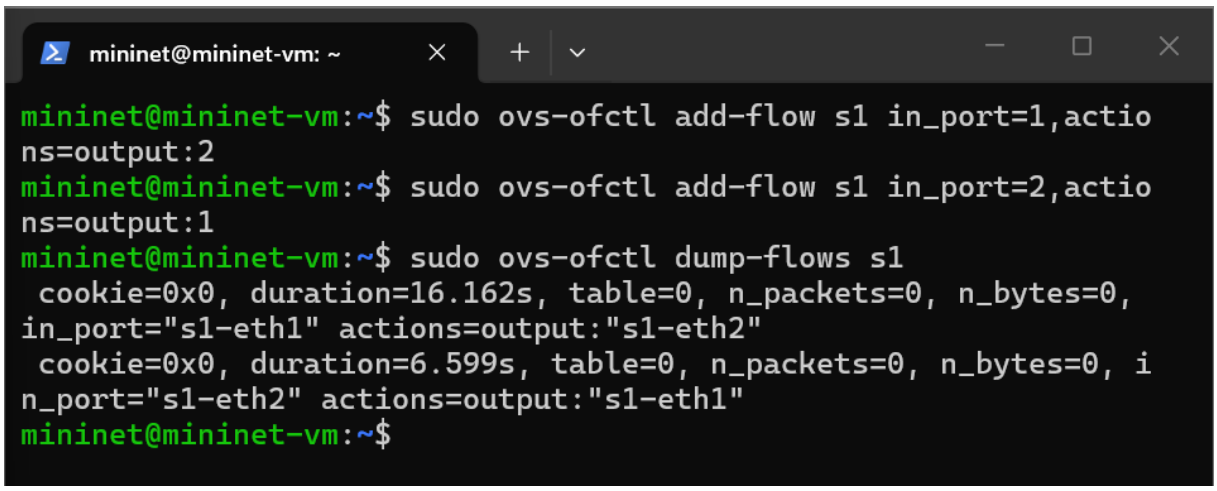


Figure 33 – Les deux flux ajoutés manuellement.

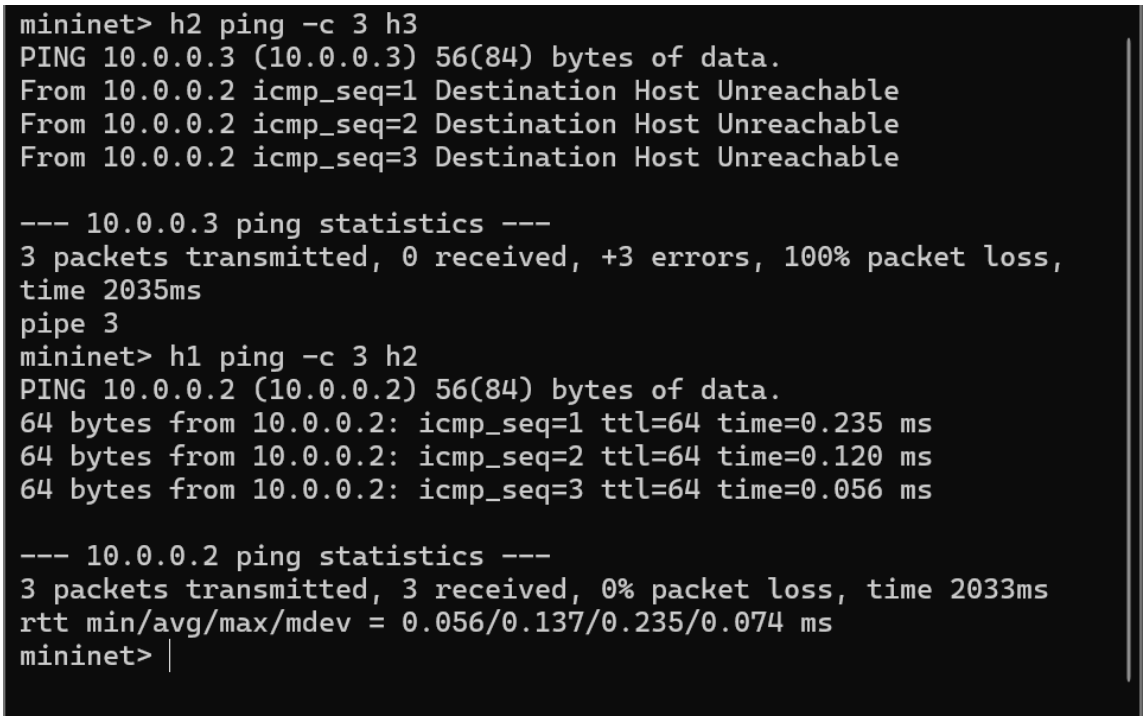


Figure 34 – Ping h2 vers h3 en échec et ping h1 vers h2 réussi.

5.3 Question 3 – Statistiques des flux

Réponse

Oui, les compteurs `n_packets` et `n_bytes` correspondent au trafic échangé entre h1 et h2 : les requêtes ARP et les trois paires de messages ICMP. Les flux ajoutés avec `ovs-ofctl` n'ont pas de délai d'expiration par défaut, donc ils restent dans la table (le sujet annonce 60 s pour son outil, ce que je n'ai pas observé ici).

Pour tester l'expiration, j'ai ajouté un flux avec `idle_timeout=120`, qui apparaît dans la table avec cette valeur, puis deux flux supplémentaires entre les ports 2 et 3 pour que h2 et h3 puissent communiquer (figures 35 et 36).

```
mininet@mininet-vm:~$ sudo ovs-ofctl dump-flows s1
 cookie=0x0, duration=87.707s, table=0, n_packets=5, n_bytes=378
, in_port="s1-eth1" actions=output:"s1-eth2"
 cookie=0x0, duration=78.144s, table=0, n_packets=8, n_bytes=504
, in_port="s1-eth2" actions=output:"s1-eth1"
mininet@mininet-vm:~$
```

Figure 35 – Compteurs des flux après le ping entre h1 et h2.

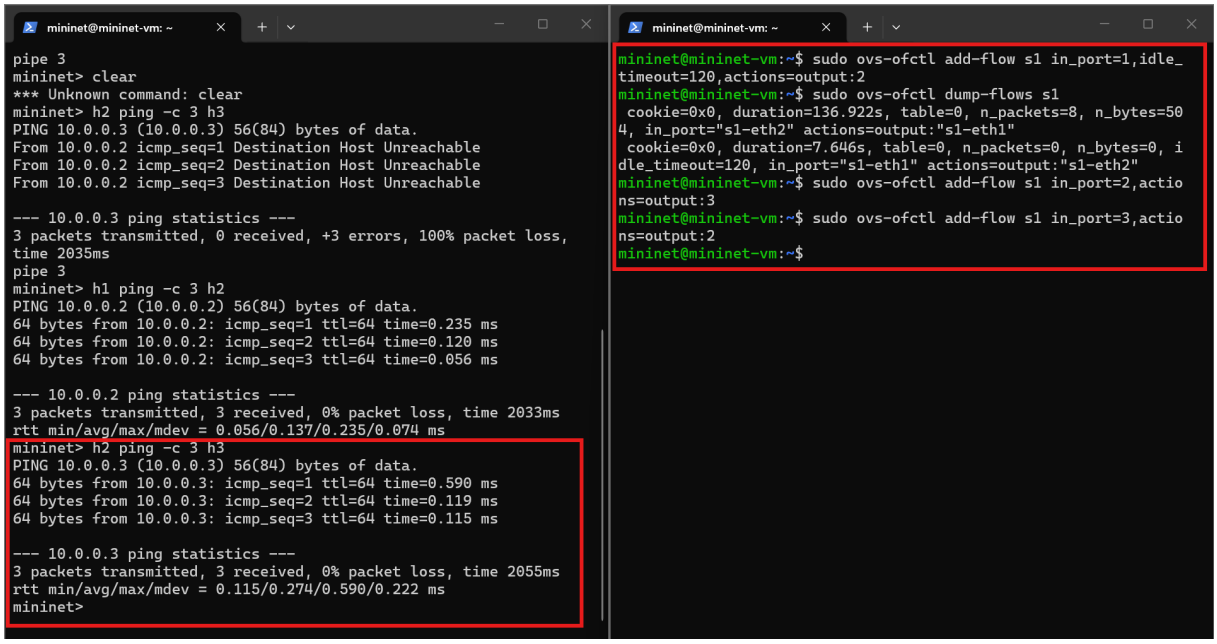


Figure 36 – Flux avec `idle_timeout=120` et ping h2 vers h3 qui réussit après l'ajout des flux.

5.4 ovs-dpctl

La commande `ovs-dpctl show` décrit le datapath noyau et ses ports, et `ovs-dpctl dump-flows` affiche les flux que le noyau a réellement en cache après le trafic. C'est une vue plus bas niveau que les règles OpenFlow : le noyau y garde des entrées très précises, créées à partir de ces règles.

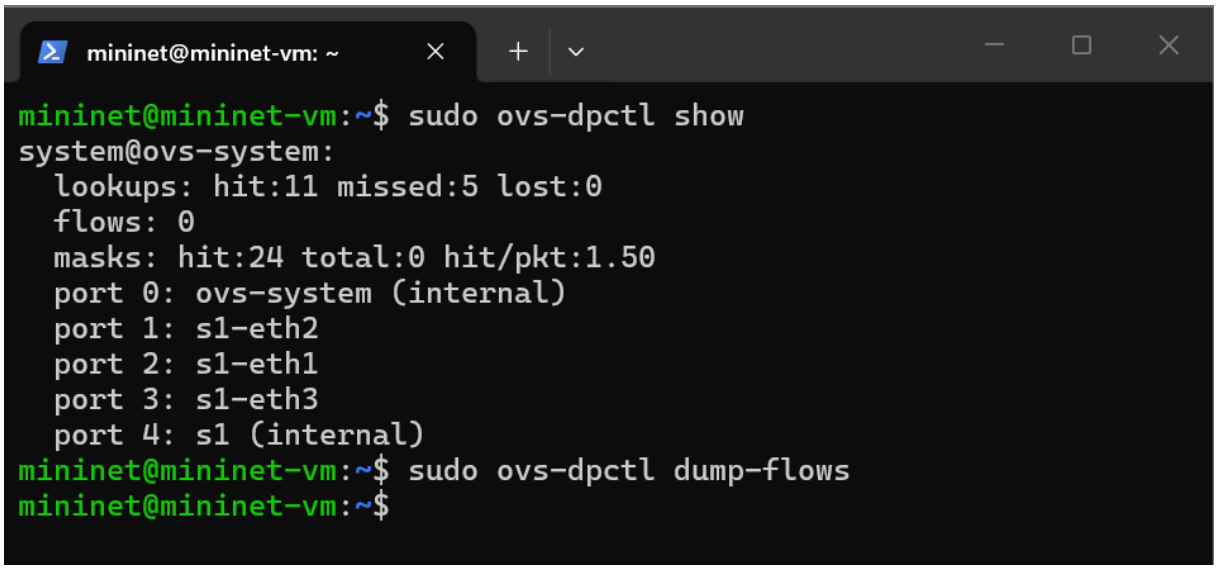


Figure 37 – Sorties de `ovs-dpctl show` et `ovs-dpctl dump-flows`.

6 Exercice 5 – Découverte de topologie

6.1 Topologie

Réponse

```
sudo mn --topo linear,4 --mac --controller remote --switch ovsk,protocols=
    OpenFlow13
```

Le texte du sujet parle de quatre switches et quatre hôtes, alors que la figure en montre cinq. J'ai suivi le texte. Pour cinq switches, il suffit de remplacer `linear,4` par `linear,5`. La commande `ovs-vsctl show` liste les quatre ponts (figure 38).

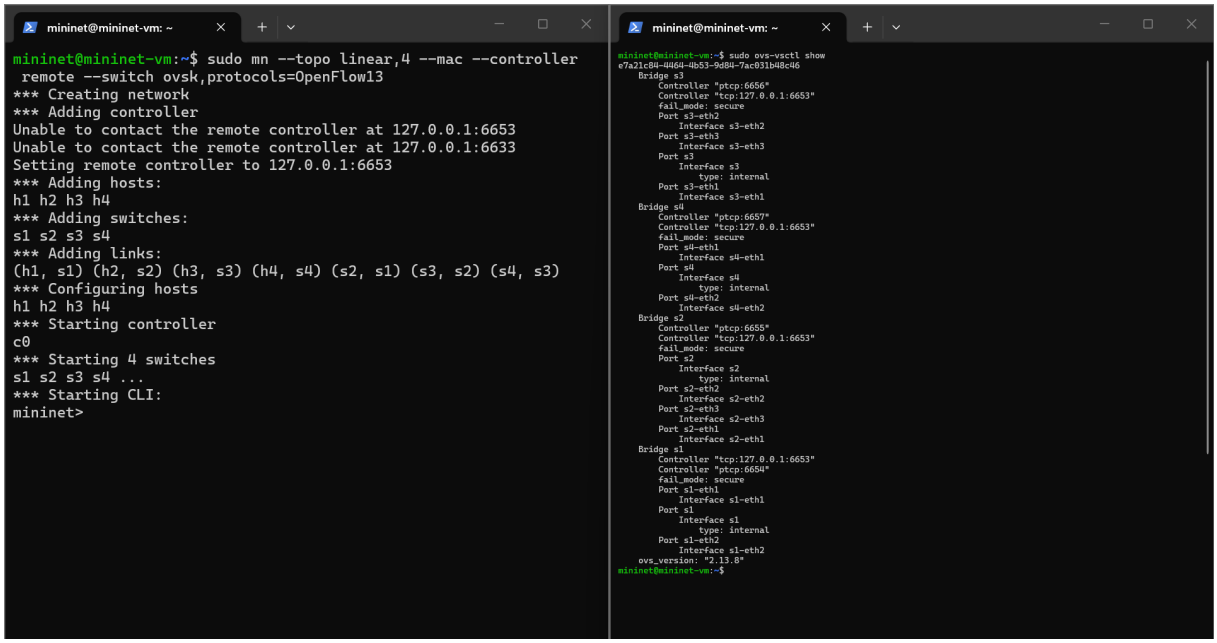


Figure 38 – Commande de déploiement et liste des switches avec `ovs-vsctl show`.

6.2 OpenDaylight

J'ai installé la version Boron-SR3 du sujet, qui demande Java 8 (`openjdk-8-jre-headless`). Dans la console Karaf, j'ai installé les fonctionnalités `odl-restconf`, `odl-l2switch-switch`, `odl-mdsal-apidocs` et `odl-dlux-core` (figure 39). Le shell Karaf n'a pas l'option `-E` de `grep`, j'ai donc vérifié chaque fonctionnalité séparément. Avec Mininet démarré, `pingall` fonctionne une fois les hôtes découverts (figure 40), et l'interface web (`http://<IP de la VM>:8181/index.html`, compte admin) affiche les switches (figure 41).



Figure 39 – Fonctionnalités ODL installées dans Karaf.

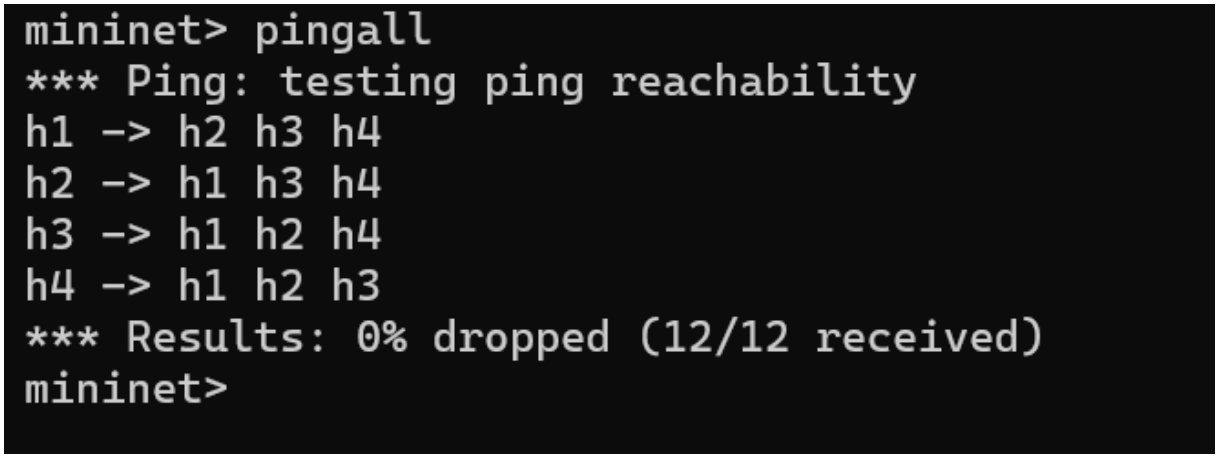


Figure 40 – `pingall` avec OpenDaylight.

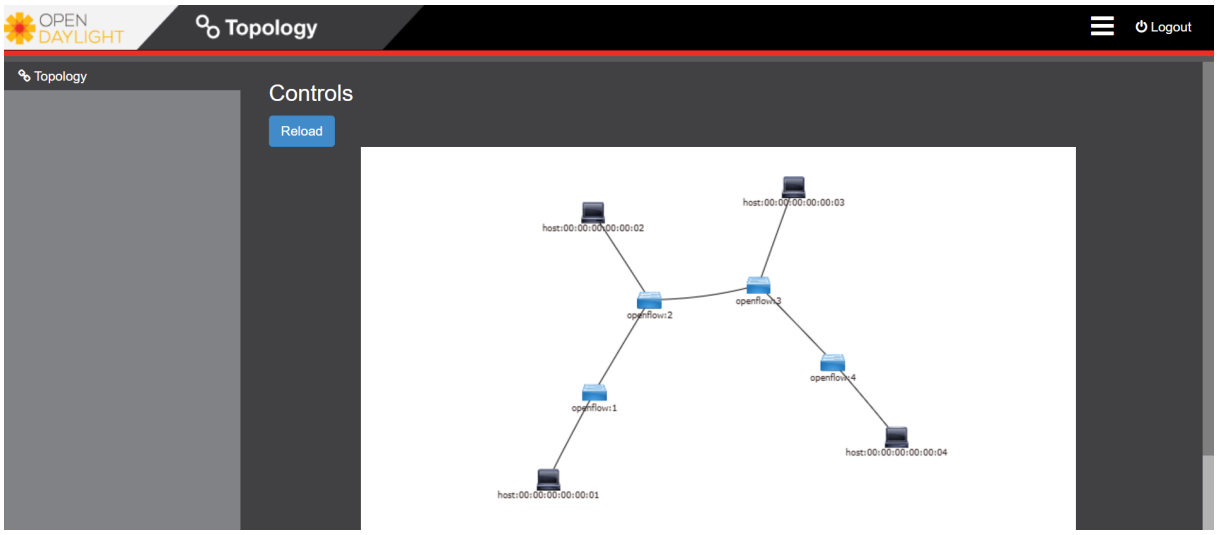


Figure 41 – Topologie affichée dans l'interface web d'OpenDaylight.

6.3 ONOS

Le disque de la VM (7,4 Go) était trop petit pour l'image ONOS : j'ai agrandi le disque de la VM à 16 Go, puis agrandi la partition et le système de fichiers. ONOS tourne dans un conteneur Docker (`docker run -t -d --net=host --name onos onosproject/onos`). Depuis la console ONOS, j'ai activé `org.onosproject.openflow` et `org.onosproject.fwd` (nécessaire pour que `pingall` fonctionne) (figure 42). `pingall` fonctionne (figure 43) et l'interface web (`http://<IP de la VM>:8181/onos/ui`) montre la topologie (figure 44).

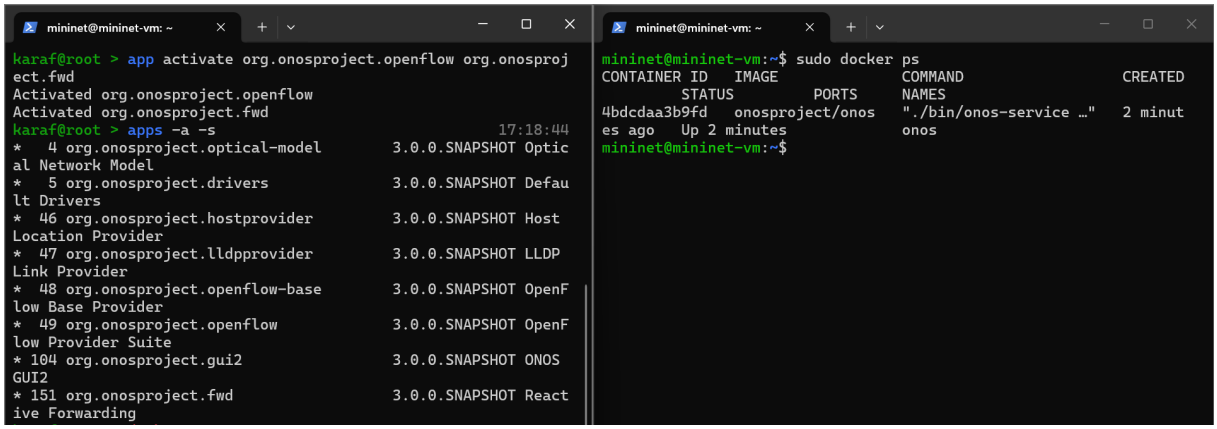


Figure 42 – Conteneur ONOS en fonctionnement et applications actives.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet>
```

Figure 43 – `pingall` avec ONOS.

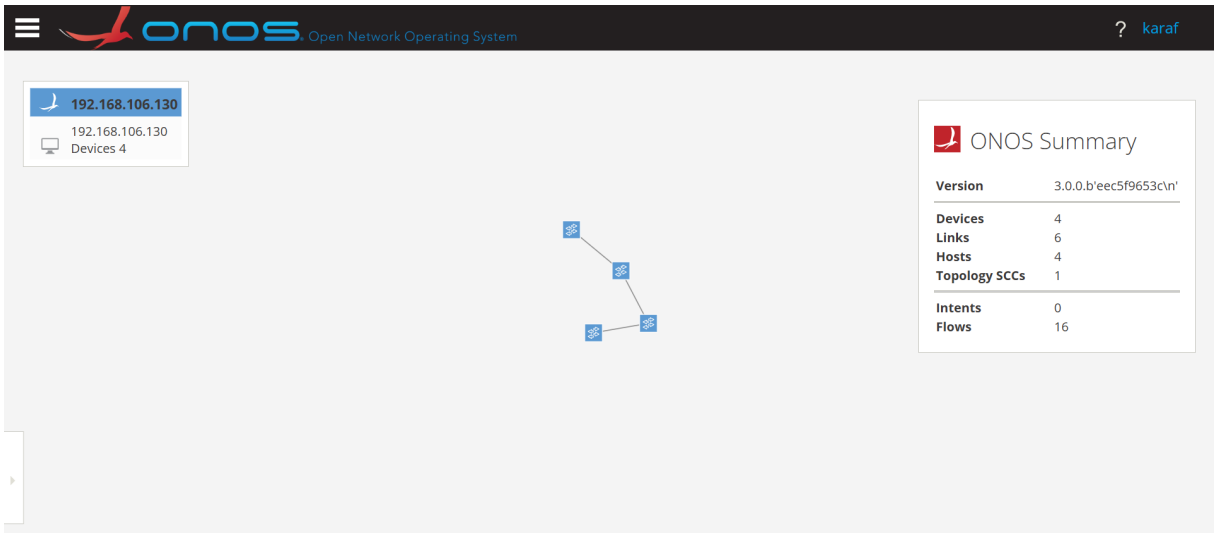


Figure 44 – Topologie affichée dans l'interface web d'ONOS.

6.4 Question 2 – Ethertype et adresse MAC destination

Réponse

Les trames LLDP portent l'ethertype `0x88cc`. Avec OpenDaylight, l'adresse destination est `01:23:00:00:00:01` : une adresse multicast (bit de groupe à 1) propre à ODL, différente de l'adresse LLDP standard `01:80:c2:00:00:0e`. Avec ONOS, les trames LLDP vont vers `01:80:c2:00:00:0e`, et ONOS envoie aussi des trames BDDP, d'ethertype `0x8942` et d'adresse destination de broadcast `ff:ff:ff:ff:ff:ff`.

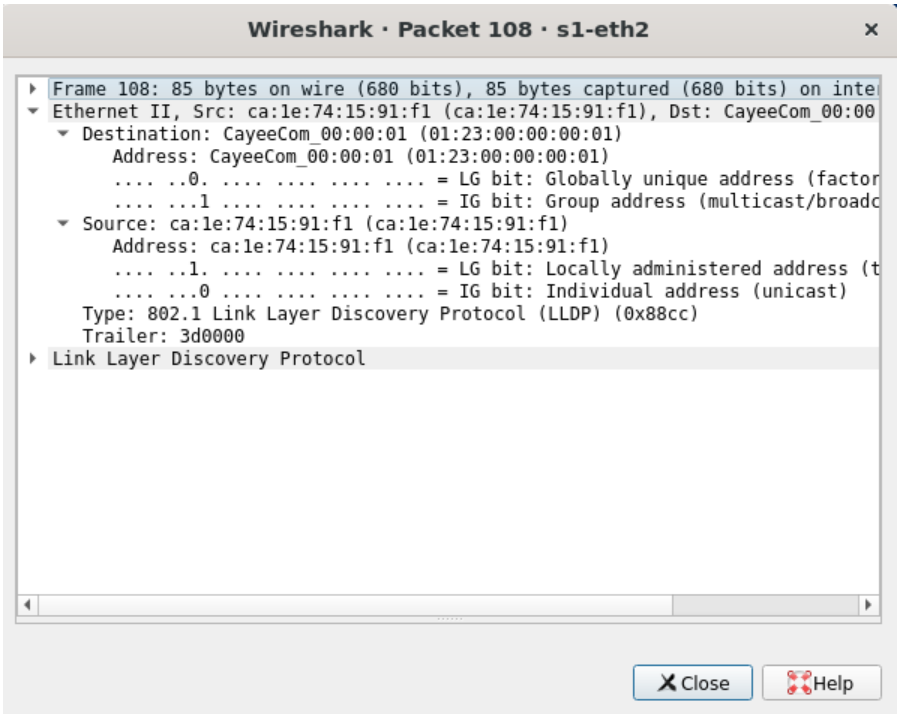


Figure 45 – Trame LLDP envoyée par OpenDaylight : ethertype et adresse destination.

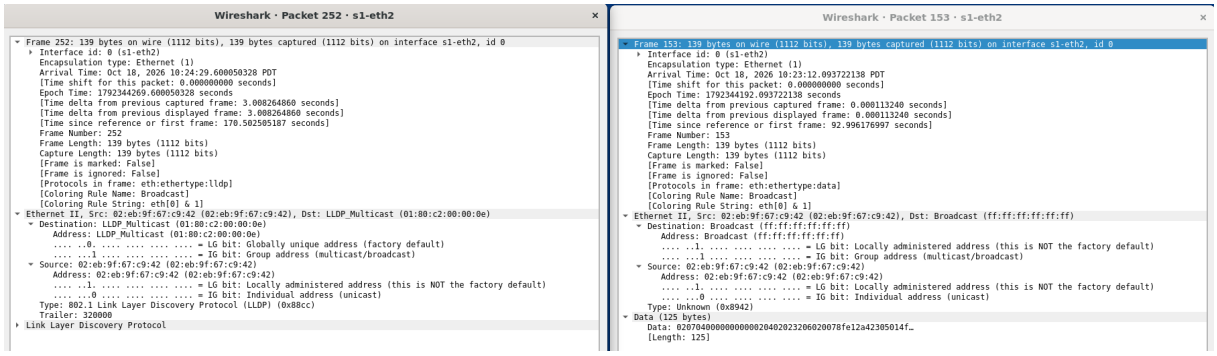


Figure 46 – Trames LLDP et BDDP envoyées par ONOS.

6.5 Question 3 – TLV

Réponse

Une trame LLDP contient une suite de TLV (Type, Longueur, Valeur) : le type dit de quelle information il s'agit, la longueur donne la taille de la valeur en octets, et la valeur contient la donnée. Les TLV obligatoires sont : *Chassis ID* (identifiant du switch), *Port ID* (port d'émission), *Time To Live* (durée de validité, ici 120 secondes) et *End of LLDPDU*. Les trames observées en contiennent 5 au total : OpenDaylight et ONOS ajoutent leurs propres TLV (ONOS y met par exemple l'identifiant `of:...` du switch).

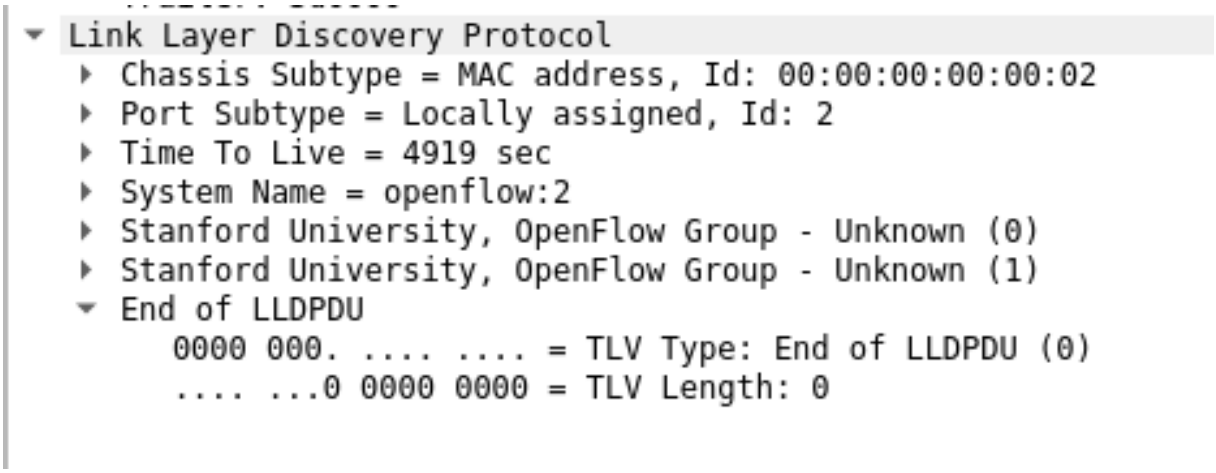


Figure 47 – TLV d'une trame LLDP d'OpenDaylight.

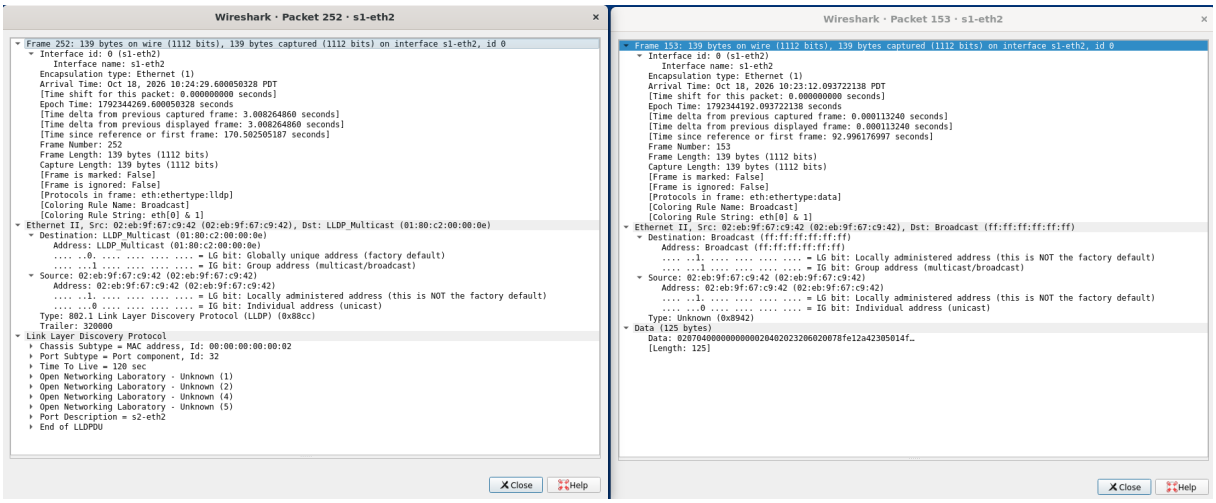


Figure 48 – TLV d'une trame LLDP d'ONOS.

6.6 Question 4 – Packet-In et Packet-Out

Réponse

`PACKET_OUT` : message du contrôleur vers le switch, qui lui demande d'émettre un paquet sur un port donné. Il contient l'action (le port de sortie) et le paquet à envoyer, ici la trame LLDP. `PACKET_IN` : message du switch vers le contrôleur, qui lui remonte un paquet reçu. Il contient le port d'entrée, la raison de l'envoi (règle vers le contrôleur ou absence de règle) et le paquet lui-même, ici la trame LLDP reçue d'un voisin.

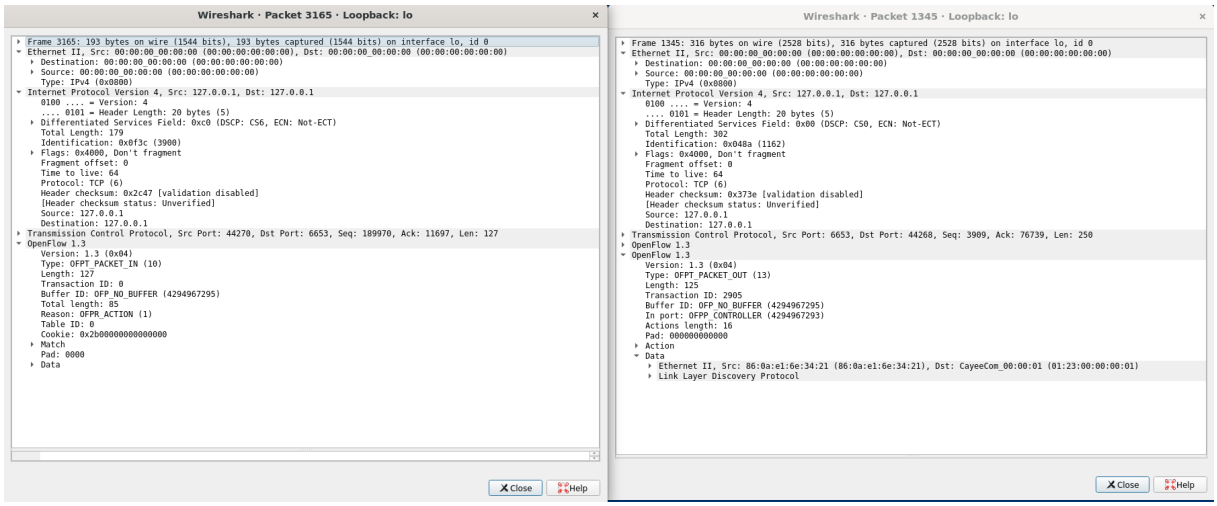


Figure 49 – Packet-Out et Packet-In transportant du LLDP avec OpenDaylight.

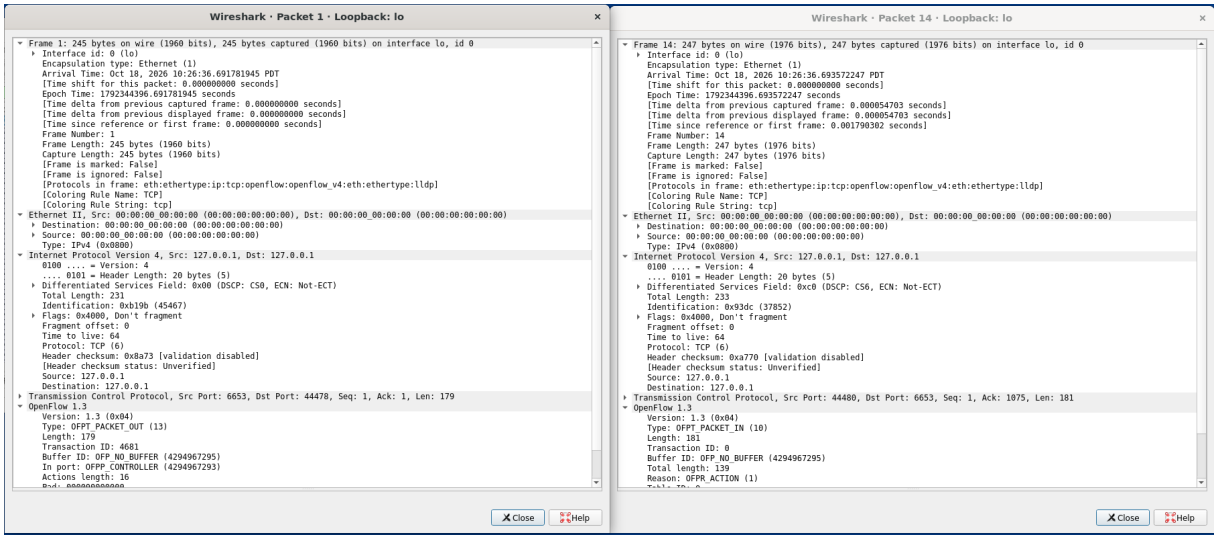


Figure 50 – Packet-Out et Packet-In transportant du LLDP avec ONOS.

6.7 Question 5 – Fréquence d'envoi du LLDP

Réponse

Avec OpenDaylight, j'ai mesuré une trame LLDP environ toutes les 5 secondes. Avec ONOS, le contrôleur émet une série de Packet-Out environ toutes les 3,1 secondes (figure 52). Raccourcir l'intervalle permet de détecter plus vite une panne de lien, mais augmente le trafic de contrôle et la charge des switches et du contrôleur. L'allonger réduit cette charge, mais ralentit la détection des pannes et laisse la vue de la topologie périmée plus longtemps.

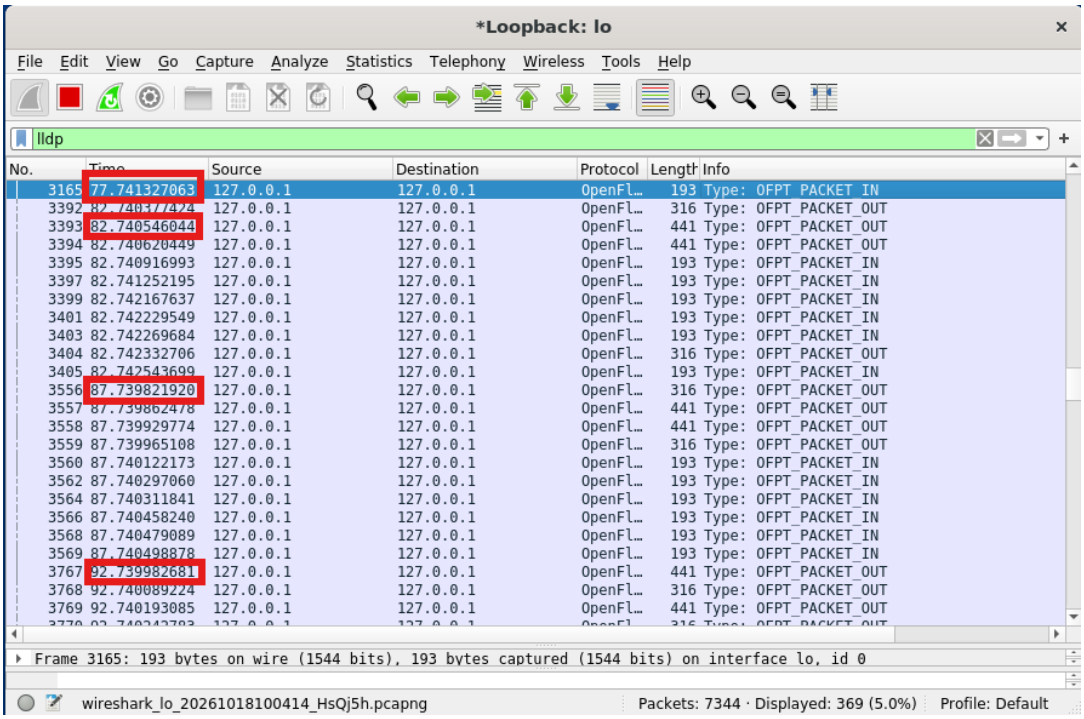


Figure 51 – Intervalle entre les trames LLDP avec OpenDaylight.

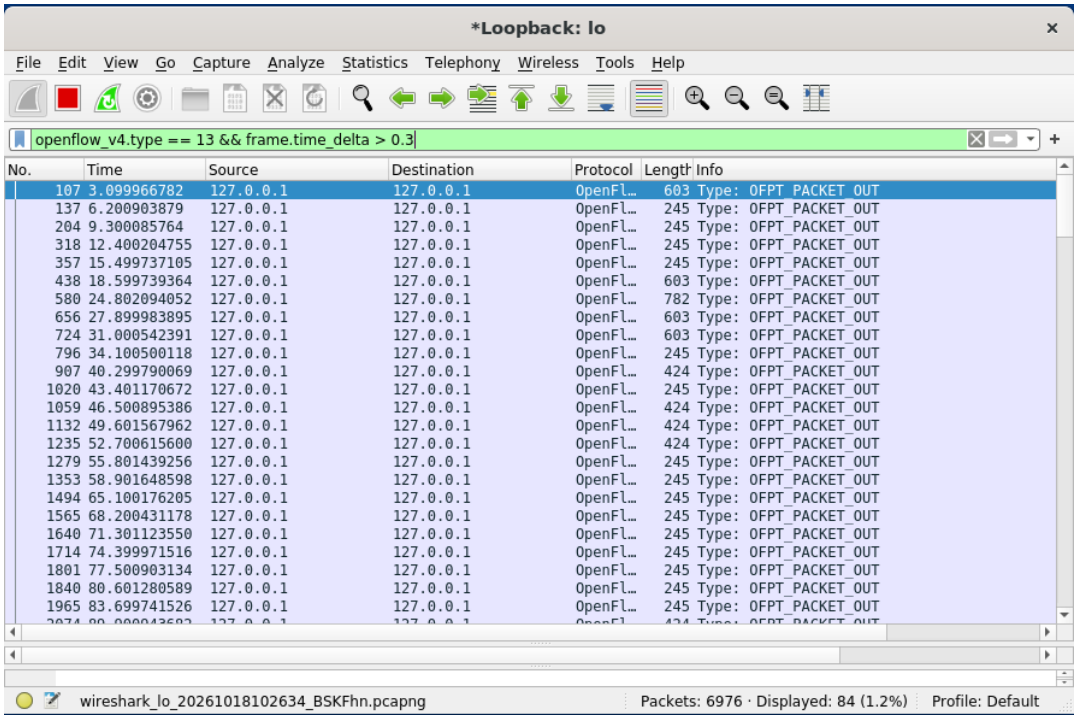


Figure 52 – Intervalle entre les séries de Packet-Out avec ONOS.

6.8 Question 6 – Comment le contrôleur découvre la topologie

Réponse

Le contrôleur envoie régulièrement, sur chaque port de chaque switch, un `PACKET_OUT` qui contient une trame LLDP indiquant le switch et le port d'émission. Le switch voisin reçoit cette trame et, comme une règle lui impose d'envoyer le LLDP au contrôleur (voir exercice 6), il la renvoie dans un `PACKET_IN`. Le contrôleur sait alors quel switch et quel port ont émis la trame, et quel switch et quel port l'ont reçue : il en déduit un lien. En répétant l'opération sur tous les ports, il reconstruit toute la topologie. Les hôtes sont découverts à part, par les ARP qui remontent au contrôleur.

7 Exercice 6 – Règles de commutation

7.1 Deux switches en OpenFlow 1.3

Réponse

```
sudo mn --topo linear,2 --controller remote --switch ovsk,protocols=OpenFlow13
```

Ce que Wireshark appelle `openflow_v4` est OpenFlow 1.3 (le numéro de version dans l'en-tête est 4).

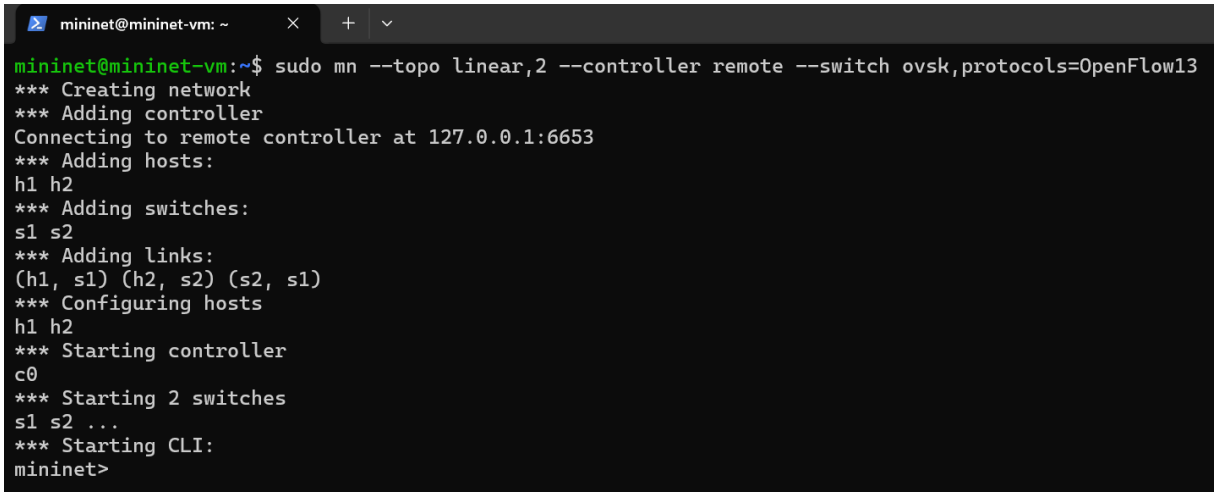


Figure 53 – Démarrage de la topologie à deux switches en OpenFlow 1.3.

J'ai arrêté la capture après quelques secondes et isolé une connexion avec *Follow TCP Stream* (flux 12 : le switch s2 et ONOS, figure 54).

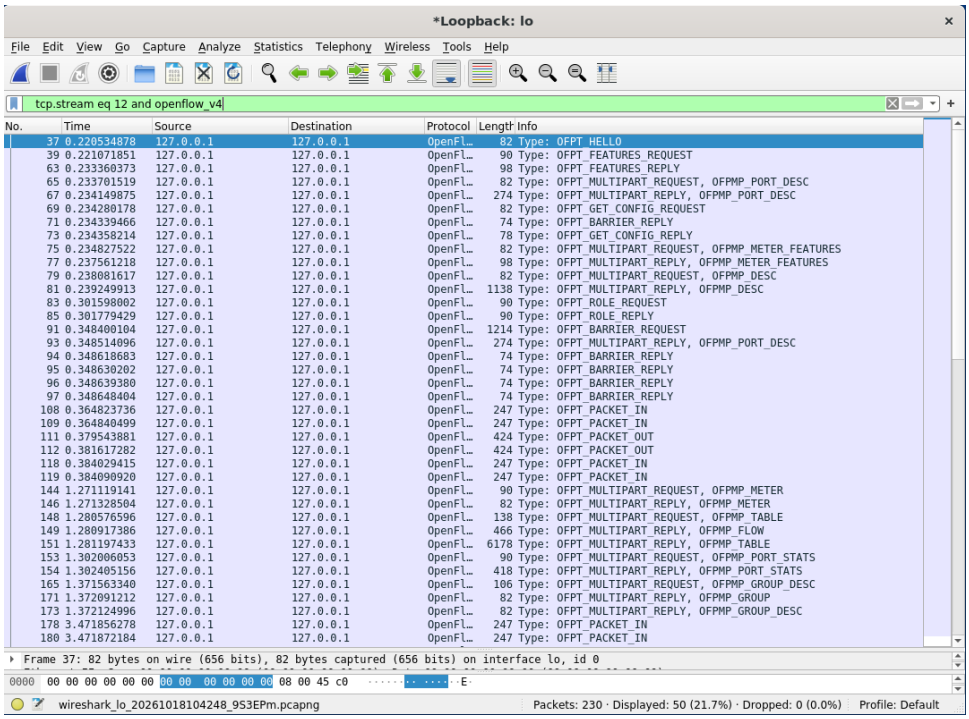


Figure 54 – Échanges OpenFlow entre un switch (s2) et le contrôleur.

Réponse

Résumé des échanges, dans l'ordre : `HELLO` ; `FEATURES_REQUEST` et `REPLY` ; `MULTIPART` pour la description des ports, puis du switch et des capacités des compteurs (`METER`) ; `GET_CONFIG` ; `ROLE_REQUEST` et `REPLY`, par lesquels ONOS se déclare maître du switch ; `BARRIER_REQUEST` et `REPLY`, qui garantissent que les messages précédents sont traités ; `PACKET_OUT` et `PACKET_IN` pour la découverte LLDP ; et des requêtes `MULTIPART` périodiques (tables, flux, ports, groupes) pour les statistiques.

Une trame TCP peut contenir plusieurs messages OpenFlow. La trame 91 en contient treize, dont des `FLOW_MOD` suivis d'un `BARRIER_REQUEST`. La colonne Info n'affiche que le dernier message de la trame, ce qui explique qu'elle semble ne contenir qu'un `BARRIER_REQUEST` (figure 55).

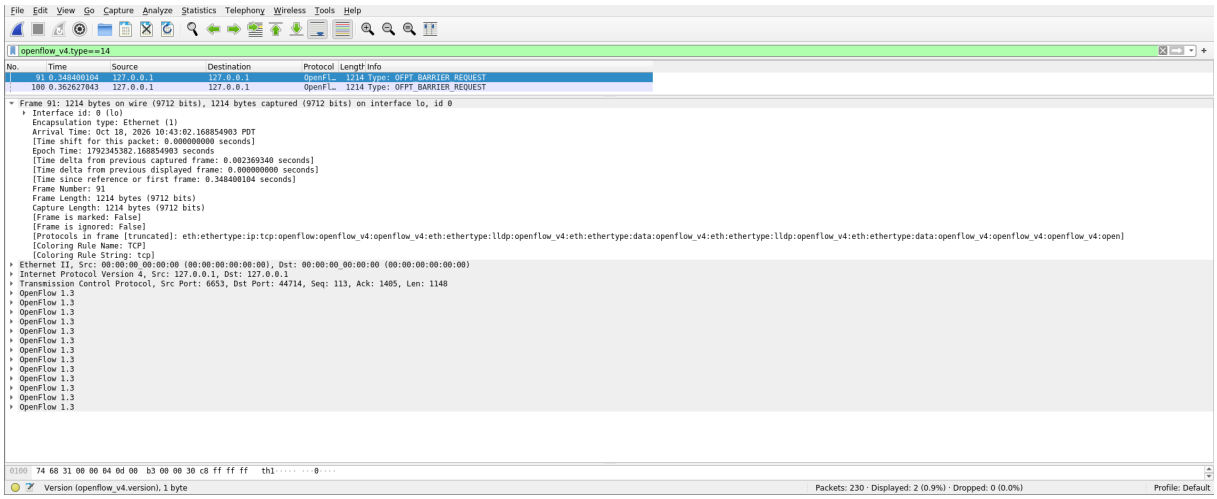


Figure 55 – Une trame contenant treize messages OpenFlow.

7.2 Messages de type 14

Réponse

Les paquets de type 14 sont des `OFPT_FLOW_MOD` : le contrôleur s'en sert pour ajouter, modifier ou supprimer des entrées dans la table de flux d'un switch.

Celui de la figure 56 est un ajout (`OFPFC_ADD`) de priorité 40000, sans délai d'expiration (idle et hard à 0). Son identifiant (*cookie*) est le même que celui de la règle BDDP visible plus loin sur le switch, c'est donc le message qui l'a installée.

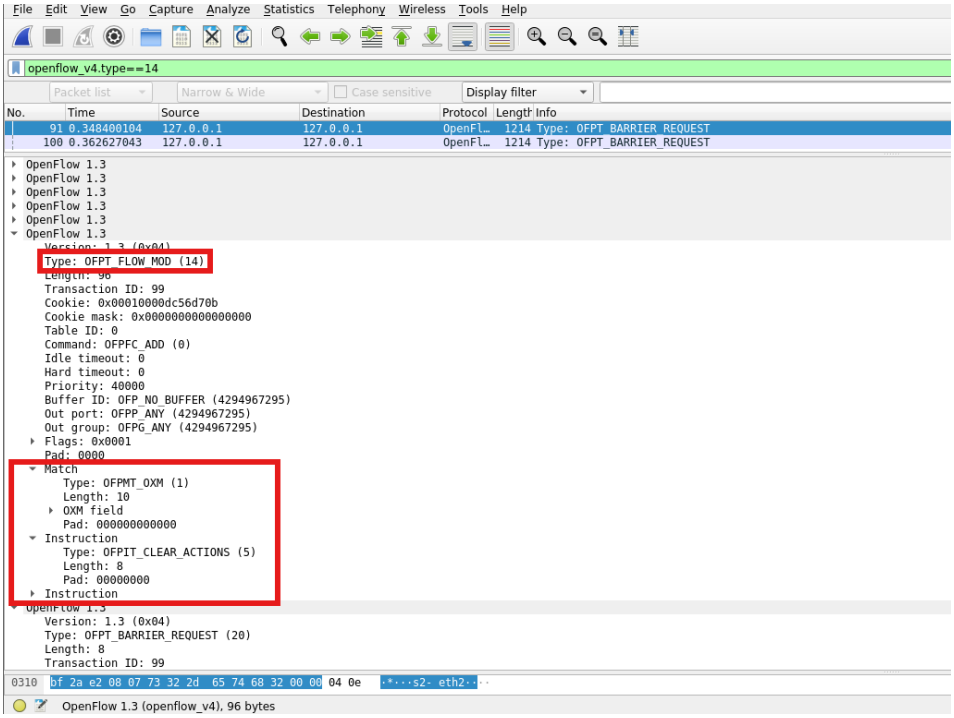


Figure 56 – Message `FLOW_MOD` : commande, priorité, match et instructions.

7.3 Règles par défaut de s2

Sur la topologie de l'exercice 5, `ovs-ofctl dump-flows s2 -O OpenFlow13` affiche quatre règles (figure 57). L'option `-O OpenFlow13` est nécessaire car le pont ne parle qu'OpenFlow 1.3.

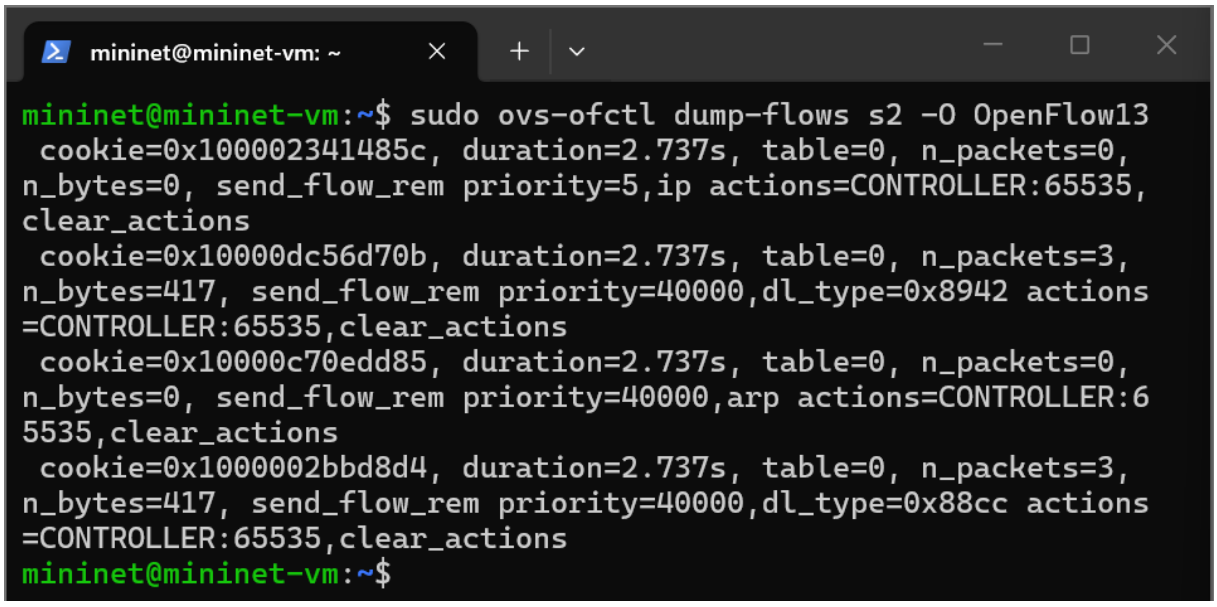


Figure 57 – Règles installées par ONOS sur s2 avant tout trafic.

Réponse

Priorité	Match	Rôle
40000	`dl_type=0x88cc`	trames LLDP envoyées au contrôleur (découverte des liens)
40000	`dl_type=0x8942`	trames BDDP envoyées au contrôleur
40000	`arp`	requêtes et réponses ARP envoyées au contrôleur (découverte des hôtes)
5	`ip`	paquets IPv4 envoyés au contrôleur

Toutes ces règles ont pour action `CONTROLLER:65535` (le paquet complet va au contrôleur). Il n'y a pas de règle `table-miss` de priorité 0, contrairement à Ryu : tout le reste est abandonné. Les priorités très élevées garantissent que la découverte passe avant toute règle de trafic. Comme ces règles n'ont pas de délai d'expiration, elles sont permanentes.

7.4 IDLE_TIMEOUT et HARD_TIMEOUT

Réponse

L'`idle_timeout` supprime le flux après N secondes *sans paquet correspondant* : un trafic régulier le maintient en vie. Le `hard_timeout` le supprime N secondes après son installation, *quoi qu'il arrive*. La valeur 0 signifie « pas d'expiration » : c'est le cas des règles de découverte d'ONOS (S56, S57). Les flux de transfert que l'application `fwd` d'ONOS installe après les pings expirent après environ 10 secondes d'inactivité ; Ryu, lui, installe ses flux sans délai.

7.5 Pings et nouveaux flux

En capturant avec le filtre `icmp || openflow_v4` pendant un `pingall`, on ne voit pas de paquet ICMP indépendant sur `lo` : les hôtes communiquent par leurs liens virtuels. Wireshark décode l'ICMP à l'intérieur des `PACKET_IN` et `PACKET_OUT` qui le transportent (figure 58).

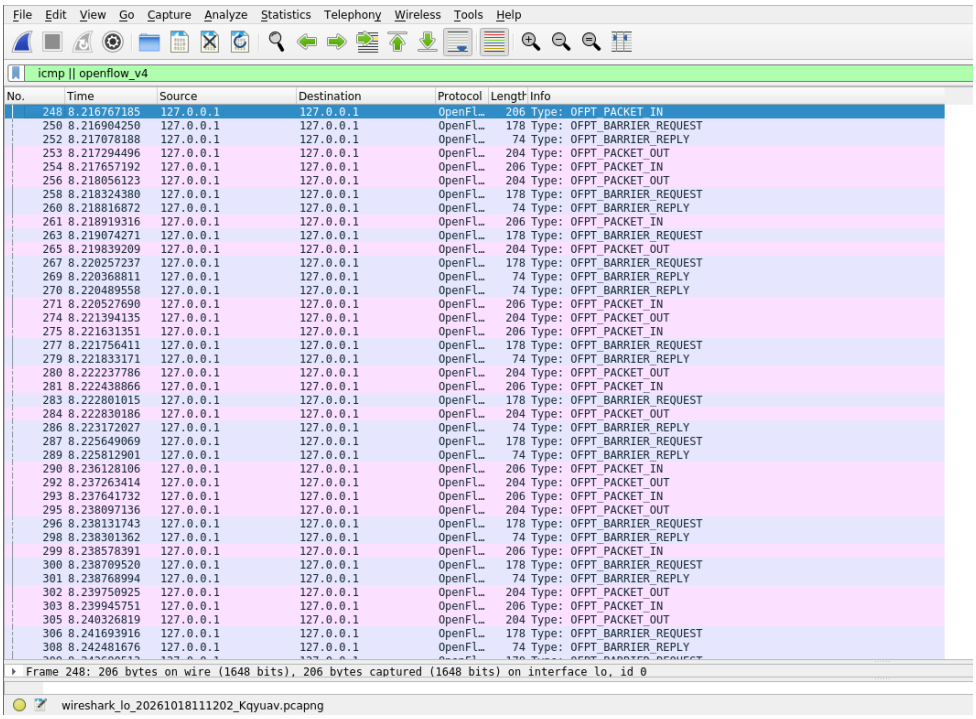


Figure 58 – Capture pendant `pingall` : Packet-In, Packet-Out et flux installés.

Réponse

Pour chaque première communication entre deux hôtes, le switch envoie le paquet au contrôleur (`PACKET_IN`), qui le renvoie (`PACKET_OUT`) et installe des flux (`FLOW_MOD` suivi d'un `BARRIER_REQUEST`). Les pings suivants entre les mêmes hôtes correspondent à ces flux et ne génèrent plus de trafic de contrôle : c'est le comportement réactif de l'application `fwd`.

Réponse

Oui, de nouveaux flux apparaissent sur s2 juste après le `pingall` (figure 59), avec un `idle_timeout`. s2 est au milieu de la ligne : les pings entre les hôtes situés de part et d'autre passent par lui, et l'application `fwd` installe des flux sur chaque switch du chemin. Ils disparaissent quelques secondes après la fin du trafic, il faut donc afficher la table tout de suite.

```
mininet@mininet-vm:~$    sudo ovs-ofctl dump-flows s2 -O OpenFlow13
 cookie=0x100002341485c, duration=493.173s, table=0, n_packets=31, n_bytes=3038, send_flow_rem priority=5,ip actions=CONTROLLER:65535,clear_actions
 cookie=0x10000dc56d70b, duration=493.173s, table=0, n_packets=319, n_bytes=44341, send_flow_rem priority=40000,dl_type=0x8942 actions=CONTROLLER:65535,clear_actions
 cookie=0x10000c70edd85, duration=493.173s, table=0, n_packets=31, n_bytes=1302, send_flow_rem priority=40000,arp actions=CONTROLLER:65535,clear_actions
 cookie=0x1000002bbd8d4, duration=493.173s, table=0, n_packets=319, n_bytes=44341, send_flow_rem priority=40000,dl_type=0x88cc actions=CONTROLLER:65535,clear_actions
 cookie=0x9700008417e3d0, duration=1.454s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth2",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:02
 actions=output:"s2-eth1"
 cookie=0x97000007e5a9e9, duration=1.452s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth1",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:01
 actions=output:"s2-eth2"
 cookie=0x970000664b01ce, duration=1.443s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth2",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03
 actions=output:"s2-eth3"
 cookie=0x970000c204a428, duration=1.436s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth3",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01
 actions=output:"s2-eth2"
 cookie=0x970000b7402ef5, duration=1.430s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth2",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:04
 actions=output:"s2-eth3"
 cookie=0x970000c33470bb, duration=1.422s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth3",dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:01
 actions=output:"s2-eth2"
 cookie=0x9700007c402bce, duration=1.412s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth1",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:03
 actions=output:"s2-eth3"
 cookie=0x9700009af305a1, duration=1.409s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth3",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:02
 actions=output:"s2-eth1"
 cookie=0x970000f504a658, duration=1.401s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth1",dl_src=00:00:00:00:00:02,dl_dst=00:00:00:00:00:04
 actions=output:"s2-eth3"
 cookie=0x9700004cd81cfb, duration=1.398s, table=0, n_packets=1, n_bytes=98, send_flow_rem priority=10,in_port="s2-eth3",dl_src=00:00:00:00:00:04,dl_dst=00:00:00:00:00:02
 actions=output:"s2-eth1"
mininet@mininet-vm:~$
```

Figure 59 – Règles sur s2 juste après `pingall` : nouveaux flux de transfert.

7.6 Compteurs sur s1 et ping vers une adresse inexistante

Comme la session SSH n'a pas d'affichage, la commande `xterm h1` du sujet échoue (*Cannot connect to display*). J'ai lancé le même ping directement depuis le CLI de Mininet, avec `h1 ping -c 8 10.0.0.10`, ce qui donne le même résultat. Les figures 60 et 62 montrent la table de s1 avant et après (figure 61 : le ping, sans réponse).

```
mininet@mininet-vm:~$ sudo ovs-ofctl dump-flows s1 -O OpenFlow13
 cookie=0x10000ea6f4b8e, duration=525.082s, table=0, n_packets=31, n_bytes=1302, send_flow_rem priority=40000,arp actions=CONTROLLER:65535,clear_actions
 cookie=0x10000021b41dc, duration=525.082s, table=0, n_packets=19, n_bytes=1862, send_flow_rem priority=5,ip actions=CONTROLLER:65535,clear_actions
 cookie=0x100009465555a, duration=525.082s, table=0, n_packets=171, n_bytes=23769, send_flow_rem priority=40000,dl_type=0x88cc actions=CONTROLLER:65535,clear_actions
 cookie=0x100007a585b6f, duration=525.082s, table=0, n_packets=171, n_bytes=23769, send_flow_rem priority=40000,dl_type=0x8942 actions=CONTROLLER:65535,clear_actions
mininet@mininet-vm:~$
```

Figure 60 – Règles de s1 avant le ping vers 10.0.0.10.

```
mininet@mininet-vm:~$    sudo ovs-ofctl dump-flows s1 -O OpenFlow13
 cookie=0x10000ea6f4b8e, duration=737.927s, table=0, n_packets=40, n_bytes=1680, send_flow_rem priority=40000,arp actions=CONTROLLER:
65535,clear_actions
 cookie=0x10000021b41dc, duration=737.927s, table=0, n_packets=19, n_bytes=1862, send_flow_rem priority=5,ip actions=CONTROLLER:65535
,clear_actions
 cookie=0x100009465555a, duration=737.927s, table=0, n_packets=239, n_bytes=33221, send_flow_rem priority=40000,dl_type=0x88cc action
s=CONTROLLER:65535,clear_actions
 cookie=0x100007a585b6f, duration=737.927s, table=0, n_packets=239, n_bytes=33221, send_flow_rem priority=40000,dl_type=0x8942 action
s=CONTROLLER:65535,clear_actions
mininet@mininet-vm:~$
```

Figure 61 – Ping vers l'adresse inexistante 10.0.0.10 : aucune réponse.

```
mininet>    h1 ping -c 8 10.0.0.10
PING 10.0.0.10 (10.0.0.10) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable
From 10.0.0.1 icmp_seq=3 Destination Host Unreachable
From 10.0.0.1 icmp_seq=4 Destination Host Unreachable
From 10.0.0.1 icmp_seq=5 Destination Host Unreachable
From 10.0.0.1 icmp_seq=6 Destination Host Unreachable
From 10.0.0.1 icmp_seq=7 Destination Host Unreachable
From 10.0.0.1 icmp_seq=8 Destination Host Unreachable

--- 10.0.0.10 ping statistics ---
8 packets transmitted, 0 received, +8 errors, 100% packet loss, time 7152ms
pipe 4
mininet>
```

Figure 62 – Règles de s1 après le ping vers 10.0.0.10.

Réponse

Règle de s1	Avant	Après	Variation
`arp` (priorité 40000)	31 paquets, 1302 octets	40 paquets, 1680 octets	**+9 paquets**
`ip` (priorité 5)	19	19	0
LLDP (`0x88cc`)	171	239	+68
BDDP (`0x8942`)	171	239	+68

Le ping correspond à la règle `arp` (priorité 40000). Personne ne possède l'adresse 10.0.0.10 : h1 diffuse des requêtes ARP, environ une par seconde, et ne reçoit jamais de réponse. Les 378 octets ajoutés correspondent à 9 trames ARP de 42 octets. Sans réponse ARP, h1 n'envoie aucun paquet ICMP, donc la règle `ip` ne bouge pas. L'augmentation des compteurs LLDP et BDDP vient de la découverte de topologie qui continue en arrière-plan : une trame toutes les 3 secondes environ, ce qui correspond à la période mesurée.

8 Adaptations et difficultés

Élément du sujet	Problème	Solution retenue
`python2-ryu`	Python 2 n'existe plus sur la VM	`python3-ryu` avec `apt-get`
`dpctl ... :6634`	Connexion refusée avec Open vSwitch	`ovs-ofctl` (et `dpctl` dans le CLI)
`xterm h1`	Pas d'affichage dans une session SSH	`h1 ping` directement dans le CLI Mininet
OpenDaylight Boron-SR3	Nécessite Java 8	`openjdk-8-jre-headless`
Contrôleur ONOS	Disque de 7,4 Go trop petit pour l'image	Disque agrandi à 16 Go, ONOS dans Docker
Port du contrôleur	Les contrôleurs écoutent sur 6653	Port 6653 utilisé partout

Table 1 – Commandes du sujet adaptées à l'environnement.

Deux incidents m'ont fait perdre du temps. D'abord, la capture Wireshark de la tempête de broadcast (exercice 2) a rempli le disque de la VM : il a fallu tuer Wireshark pour libérer l'espace. Ensuite, pendant l'exercice 3, Ryu a été arrêté plusieurs fois parce que `sudo mn -c` termine aussi les processus du contrôleur. J'ai donc fini par faire le nettoyage avant de lancer Ryu, et jamais après.

9 Bilan

Exercice	Contenu	Réussi	Captures
1	Arbre simple (haut et bas niveau), arrêt de c1	Oui	S02 à S10
2	Réseau avec boucle	Oui	S11 à S15
3	Messages OpenFlow 1.3 avec Ryu	Partiel	S16 à S29
4	dpctl, ovs-ofctl, ovs-dpctl	Oui	S31 à S37
5	Découverte de topologie (ODL, ONOS)	Oui	S38 à S52
6	Règles de commutation	Oui	S53 à S62

Table 2 – Récapitulatif (cohérent avec le fichier Excel remis).

L'exercice 3 est noté « partiel » parce que les messages `SET_CONFIG` (S23) et `SET_ASYNC` (S30) n'ont pas pu être observés avec la version de Ryu installée.

10 Conclusion

Ce TP m'a montré le fonctionnement concret d'un contrôleur SDN. L'exercice 3 donne la séquence complète d'une connexion OpenFlow 1.3, et l'exercice 4 montre ce qu'un switch fait quand il n'a ni contrôleur ni règle : rien. L'exercice 5 explique comment un contrôleur reconstruit la topologie avec du LLDP encapsulé dans des messages OpenFlow, et l'exercice 6 montre le coût de cette découverte : des règles permanentes de forte priorité qui renvoient tout au contrôleur.

Les exercices sur la topologie en arbre et sur la boucle rappellent aussi des limites du réseau lui-même : un arbre n'a aucune redondance, et une boucle sans spanning tree sature un réseau en quelques secondes.

A Script `simpletree_highlevel.py`

```python
#!/usr/bin/env python3
import sys
from mininet.topo import Topo
from mininet.net import Mininet
from mininet.node import OVSSwitch
from mininet.cli import CLI
from mininet.log import setLogLevel


class SimpleTree(Topo):
    def build(self, n=2):
        dpid = 1
        core = self.addSwitch('c1', dpid='%x' % dpid)
        host_id = 1
        edge_id = 1
        for a in range(1, n + 1):
            dpid += 1
            agg = self.addSwitch('a%d' % a, dpid='%x' % dpid)
            self.addLink(core, agg)
            for _ in range(n):
                dpid += 1
                edge = self.addSwitch('e%d' % edge_id, dpid='%x' % dpid)
                edge_id += 1
                self.addLink(agg, edge)
                for _ in range(n):
                    host = self.addHost('h%d' % host_id)
                    host_id += 1
                    self.addLink(edge, host)


def run(n):
    net = Mininet(topo=SimpleTree(n), switch=OVSSwitch)
    net.start()
    CLI(net)
    net.stop()


if __name__ == '__main__':
    setLogLevel('info')
    n = int(sys.argv[1]) if len(sys.argv) > 1 else int(input('n = '))
    run(n)
```

B Script `simpletree_lowlevel.py`

```python
#!/usr/bin/env python3
import sys
from mininet.net import Mininet
from mininet.node import OVSSwitch
from mininet.cli import CLI
from mininet.log import setLogLevel


def build(n):
```

```
    net = Mininet(switch=OVSSwitch)
    net.addController('c0')
    dpid = 1
    core = net.addSwitch('c1', dpid='%x' % dpid)
    host_id = 1
    edge_id = 1
    for a in range(1, n + 1):
        dpid += 1
        agg = net.addSwitch('a%d' % a, dpid='%x' % dpid)
        net.addLink(core, agg)
        for _ in range(n):
            dpid += 1
            edge = net.addSwitch('e%d' % edge_id, dpid='%x' % dpid)
            edge_id += 1
            net.addLink(agg, edge)
            for _ in range(n):
                host = net.addHost('h%d' % host_id)
                host_id += 1
                net.addLink(edge, host)
    return net


if __name__ == '__main__':
    setLogLevel('info')
    n = int(sys.argv[1]) if len(sys.argv) > 1 else int(input('n = '))
    net = build(n)
    net.start()
    CLI(net)
    net.stop()
```

C Script `loop.py`

```
#!/usr/bin/env python3
from mininet.net import Mininet
from mininet.node import OVSBridge
from mininet.cli import CLI
from mininet.log import setLogLevel


def run():
    net = Mininet(switch=OVSBridge, controller=None)
    s1 = net.addSwitch('s1')
    s2 = net.addSwitch('s2')
    s3 = net.addSwitch('s3')
    h1 = net.addHost('h1')
    h2 = net.addHost('h2')
    h3 = net.addHost('h3')
    net.addLink(h1, s1)
    net.addLink(h2, s2)
    net.addLink(h3, s3)
    net.addLink(s1, s2)
    net.addLink(s2, s3)
    net.addLink(s3, s1)
    net.start()
    CLI(net)
    net.stop()
```

```
if __name__ == '__main__':
    setLogLevel('info')
    run()
```